

Universidad del Istmo

Programacion de Computadoras · Unity 2D

CCPP

Derechito a la Iglesia

Documentacion Tecnica delCodigo

AUTORES

Jose David Navichoque Pineda

Jose Gonzalez

Mynor Barrios

Andre Gil

Videojuego Educativo · Unity 2D · C# · 2026

Universidad del Istmo

Documentacion generada automaticamente

Tabla de Contenidos

Tabla de Contenidos	2
Resumen del Proyecto	3
Estadisticas delCodigo Fuente	3
Arquitectura de Carpetas	3
Flujo General del Juego	3
Colisiones	5
Configuraciones	9
Data	12
Enumeraciones	26
Estructuras	27
Interfaces	28
Managers	29
Misa	32
Misiones	37
Modelos	46

Resumen del Proyecto

CCPP — Derechito a la Iglesia es un videojuego educativo 2D desarrollado en **Unity** con **C#** como parte de un proyecto academico en la **Universidad del Istmo**. El juego ensena al jugador las posturas y gestos correctos durante una misa catolica, mediante una serie de minijuegos interactivos que evaluan el control del cuerpo (piernas, pecho, cabeza, manos, boca) en diferentes momentos de la liturgia.

La arquitectura del codigo sigue el patron **MVC (Model-View-Controller)** adaptado a Unity: las clases *Data* y *ScriptableObjects* representan el modelo, los *Controllers* implementan la logica de comportamiento y los sistemas de UI actuan como vista. Los *Managers* orquestan la comunicacion entre subsistemas mediante **eventos C# (event/delegate)**, manteniendo un bajo acoplamiento entre componentes.

Estadisticas del Codigo Fuente

59	5103	10
Archivos .cs	Lineas de codigo	Categorias

Arquitectura de Carpetas

Colisiones	3	Contiene scripts que detectan colisiones fisicas en el mundo del juego. Estos componentes se adjuntan a objetos de escen...
Configuraciones	6	ScriptableObjects de configuracion para cada minijuego. Definen parametros ajustables desde el Inspector de Unity como t...
Data	20	Clases de datos en tiempo de ejecucion (MonoBehaviours y ScriptableObjects) que almacenan el estado del jugador, la cama...
Enumeraciones	3	Enumeraciones de C# que definen conjuntos de valores con nombre. Facilitan la lectura del codigo y evitan el uso de ente...
Estructuras	2	Structs de C# que agrupan datos relacionados de forma ligera y eficiente. Se usan para representar resultados de accione...
Interfaces	1	Contratos de interfaz que definen el comportamiento minimo que deben implementar ciertas clases. Permiten el polimorfism...
Managers	2	Orquestadores centrales del juego. Escuchan eventos del sistema y delegan la logica al componente apropiado. El Minijueg...
Misa	3	Scripts especificos de la escena de Misa. Controlan el flujo narrativo de la ceremonia, la barra de vida/energia del jug...
Misiones	4	Implementaciones concretas del sistema de misiones. Cada clase extiende la base de mision y define las condiciones de in...
Modelos	15	Controladores MVC del juego. Implementan la logica de presentacion y comportamiento de cada entidad: jugador, camara, di...

Flujo General del Juego

El jugador inicia en la escena principal y avanza hacia la iglesia. Al colisionar con zonas de dialogo (*ColisionDialogo*) se activan conversaciones que bloquean el movimiento. Al llegar al altar, se activa el **GestorMisa** que orquesta las misiones (*Mision01*, *MisionMinijuego*, *MisionPosicion*, *MisionPosturaParado*). Cada mision puede requerir que el jugador complete un minijuego a traves del **MinijuegoManager**, el cual delega segun la parte del cuerpo seleccionada en el panel *CuerpoUIController*. Al completar todos los objetivos, *FinalJuego* registra el resultado y termina la partida.

Colisiones

Contiene scripts que detectan colisiones físicas en el mundo del juego. Estos componentes se adjuntan a objetos de escena y reaccionan cuando el jugador entra en contacto con ellos, disparando diálogos, efectos o el fin de la partida.

Archivo	Líneas	Ruta relativa
ColisionDialogo.cs	49	<i>colisiones/ColisionDialogo.cs</i>
FinalJuego.cs	215	<i>colisiones/FinalJuego.cs</i>
TriggerDesactivarObjeto.cs	26	<i>colisiones/TriggerDesactivarObjeto.cs</i>

ColisionDialogo.cs

49 líneas

Scripts/colisiones/ColisionDialogo.cs

Descripción	Componente de trigger que se adjunta a zonas de colisión en la escena. Cuando el jugador entra en el área (OnTriggerEnter), bloquea su movimiento mediante JugadorData.puedeMoverse y lanza una conversación a través de DialogoController. Al terminar el diálogo (evento OnDialogoTerminado) restaura el movimiento. Dependencias: JugadorData, DialogoController, DialogoData.
--------------------	---

```
using UnityEngine;

/// <summary>
/// Al colisionar con el jugador lanza un diálogo, bloquea movimiento
/// y al terminar lo reactiva. Se puede repetir al regresar.
/// </summary>
public class ColisionDialogo : MonoBehaviour
{
    [Header("Diálogo")]
    public DialogoData dialogo;
    [Header("Jugador")]
    public JugadorData jugadorData;
    private DialogoController _dialogoController;
    private bool _disparado = false;
    private void Awake()
    {
        _dialogoController = FindFirstObjectByType<DialogoController>();
        jugadorData = FindFirstObjectByType<JugadorData>();
    }
    private void OnCollisionEnter(Collision other)
    {
        if (_disparado) return;
        if (!other.gameObject.CompareTag("Player")) return;
        if (_dialogoController == null)
        {
            Debug.LogWarning("[ColisionDialogo] No se encontró DialogoController.");
            return;
        }
        _disparado = true;
        if (jugadorData != null) jugadorData.puedeMoverse = false;
        DialogoController.OnDialogoTerminado += AlTerminarDialogo;
        _dialogoController.Iniciar(dialogo);
    }
    private void AlTerminarDialogo()
    {
        DialogoController.OnDialogoTerminado -= AlTerminarDialogo;
        if (jugadorData != null) jugadorData.puedeMoverse = true;
        _disparado = false;
    }
}
```

FinalJuego.cs

215 lines

Scripts/collisiones/FinalJuego.cs

Descripcion	Gestiona la pantalla de fin de juego. Detecta la colision del jugador con la zona final, detiene el movimiento y muestra el resultado. Contiene la logica de puntuacion final y la transicion a la pantalla de creditos o reinicio. Dependencias: JugadorData, SceneManager de Unity.
--------------------	---


```

if (objetoADesactivar == null) return;
objetoADesactivar.SetActive(false);
this.enabled = false;
}
}

```

Configuraciones

ScriptableObjects de configuración para cada minijuego. Definen parámetros ajustables desde el Inspector de Unity como tiempos, puntajes, sprites y textos, separando datos de lógica.

Archivo	Líneas	Ruta relativa
ConfigCanto.cs	32	Configuraciones/ConfigCanto.cs
ConfigCerebro.cs	30	Configuraciones/ConfigCerebro.cs
ConfigPecho.cs	31	Configuraciones/ConfigPecho.cs
ConfigResponder.cs	42	Configuraciones/ConfigResponder.cs
ConfigSaludo.cs	24	Configuraciones/ConfigSaludo.cs
ConfigSenal.cs	24	Configuraciones/ConfigSenal.cs

ConfigCanto.cs

32 líneas

Scripts/Configuraciones/ConfigCanto.cs

Descripción	ScriptableObject-style configurator para el minijuego de canto. Al activarse (OnEnable) copia sus valores serializados hacia MinijuegoCantoData, separando la configuración de la lógica. Permite diseñar múltiples dificultades o variaciones del canto desde el Inspector sin tocar código. Dependencias: MinijuegoCantoData.
--------------------	---

```

using UnityEngine;

/// <summary>
/// Al activarse aplica configuración específica al minijuego de canto.
/// </summary>
public class ConfigCanto : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoCantoData data;

    [Header("Configuración")]
    public float velocidadMin = 150f;
    public float velocidadMax = 280f;
    public float posYInicial = 400f;
    public float intervaloNotas = 1.2f;
    public float margenAcierto = 60f;
    public int notasParaGanar = 8;
    public int fallosPermitidos = 3;

    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigCanto] data no asignado."); return; }
        data.velocidadMin = velocidadMin;
        data.velocidadMax = velocidadMax;
        data.posYInicial = posYInicial;
        data.intervaloNotas = intervaloNotas;
        data.margenAcierto = margenAcierto;
        data.notasParaGanar = notasParaGanar;
        data.fallosPermitidos = fallosPermitidos;
    }
}

```

ConfigCerebro.cs

30 líneas

Scripts/Configuraciones/ConfigCerebro.cs

Descripción	Configurador del minijuego de cerebro. Transfiere parámetros de velocidad de distractores, pérdida/ganancia de alpha del cerebro y tiempo de supervivencia hacia MinijuegoCerebroData. Permite balancear la dificultad por escena. Dependencias: MinijuegoCerebroData.
--------------------	--

```
using UnityEngine;

/// <summary>
/// Al activarse aplica configuración específica al minijuego de cerebro.
/// </summary>
public class ConfigCerebro : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoCerebroData data;
    [Header("Configuración")]
    public float perdidaAlpha = 0.08f;
    public float gananciaAlpha = 0.3f;
    public float velocidadMin = 60f;
    public float velocidadMax = 160f;
    public float radioColision = 50f;
    public float tiempoSobrevivir = 15f;
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigCerebro] data no asignado."); return; }
        data.perdidaAlpha = perdidaAlpha;
        data.gananciaAlpha = gananciaAlpha;
        data.velocidadMin = velocidadMin;
        data.velocidadMax = velocidadMax;
        data.radioColision = radioColision;
        data.tiempoSobrevivir = tiempoSobrevivir;
    }
}
```

ConfigPecho.cs

31 líneas

Scripts/Configuraciones/ConfigPecho.cs

Descripción	Configurador del minijuego de pecho (ritmo cardiaco). Define cuantas rondas debe completar el jugador, el rango de velocidad del círculo decreciente y el margen de acierto. Escribe directamente en MinijuegoPechoData al activarse. Dependencias: MinijuegoPechoData.
--------------------	---

```
using UnityEngine;

/// <summary>
/// Al activarse aplica configuración específica al minijuego de pecho.
/// Coloca este script en el mismo GameObject que activas para la misión.
/// </summary>
public class ConfigPecho : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoPechoData data;
    [Header("Configuración")]
    public int rondasTotales = 4;
    public float velocidadMin = 80f;
    public float velocidadMax = 180f;
    public float margenAcierto = 30f;
    public float tamanoInicial = 400f;
    public float tamanoCorazon = 100f;
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigPecho] data no asignado."); return; }
        data.rondasTotales = rondasTotales;
        data.velocidadMin = velocidadMin;
        data.velocidadMax = velocidadMax;
        data.margenAcierto = margenAcierto;
        data.tamanoInicial = tamanoInicial;
    }
}
```

```
data.tamanoCorazon = tamanoCorazon;
}
}
```

ConfigResponder.cs

42 líneas

Scripts/Configuraciones/ConfigResponder.cs

Descripcion	Configurador del minijuego de preguntas y respuestas. Además de parametros de tiempo y barra, define el contenido de los tres botones de respuesta, el índice de la respuesta correcta y el texto de ayuda. Permite tener distintas preguntas por escena de misa sin duplicar prefabs. Dependencias: MinijuegoResponderData.
--------------------	--

```
using UnityEngine;
/// <summary>
/// Al activarse aplica configuraci3n espec3fica al minijuego de responder.
/// </summary>
public class ConfigResponder : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoResponderData data;
    [Header("Configuraci3n")]
    public float tiempoLimite = 10f;
    public float anchoBarraInicial = 600f;
    public float alphaAyudaPorEspacio = 0.15f;
    public float aceleracionBarraPorEspacio = 1.5f;
    public int indiceRespuestaCorrecta = 0;
    [Header("Contenido")]
    public string textoBoton0 = "Opci3n A";
    public string textoBoton1 = "Opci3n B";
    public string textoBoton2 = "Opci3n C";
    public string textoAyudaContenido = "La respuesta es...";
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigResponder] data no asignado."); return; }
        data.tiempoLimite = tiempoLimite;
        data.anchoBarraInicial = anchoBarraInicial;
        data.alphaAyudaPorEspacio = alphaAyudaPorEspacio;
        data.aceleracionBarraPorEspacio = aceleracionBarraPorEspacio;
        data.indiceRespuestaCorrecta = indiceRespuestaCorrecta;
        data.textoAyudaContenido = textoAyudaContenido;
        data.textoBotones = new string[3]
        {
            textoBoton0,
            textoBoton1,
            textoBoton2
        };
    }
}
```

ConfigSaludo.cs

24 líneas

Scripts/Configuraciones/ConfigSaludo.cs

Descripcion	Configurador del minijuego de saludo de mano. Ajusta el tiempo limite, el tiempo de exito para mostrar la animacion de apret3n y el ancho inicial de la barra de tiempo. Dependencias: MinijuegoSaludoData.
--------------------	---

```
using UnityEngine;
/// <summary>
/// Al activarse aplica configuraci3n espec3fica al minijuego de saludo.
/// </summary>
public class ConfigSaludo : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoSaludoData data;
    [Header("Configuraci3n")]
    public float tiempoLimite = 3f;
```

```

public float tiempoExito = 1.2f;
public float anchoBarraInicial = 400f;
private void OnEnable()
{
    if (data == null) { Debug.LogWarning("[ConfigSaludo] data no asignado."); return; }
    data.tiempoLimite = tiempoLimite;
    data.tiempoExito = tiempoExito;
    data.anchoBarraInicial = anchoBarraInicial;
}
}

```

ConfigSenal.cs

24 líneas

Scripts/Configuraciones/ConfigSenal.cs

Descripción	Configurador del minijuego de senal de la cruz. Controla el tiempo limite, el ancho de la barra y el flag esCompleja que determina si se usan los botones simples (4 pasos) o los complejos (mas pasos). Dependencias: MinijuegoSenalData.
--------------------	---

```

using UnityEngine;
/// <summary>
/// Al activarse aplica configuraci3n espec3fica al minijuego de se3al.
/// </summary>
public class ConfigSenal : MonoBehaviour
{
    [Header("Referencia")]
    public MinijuegoSenalData data;
    [Header("Configuraci3n")]
    public float tiempoLimite = 5f;
    public float anchoBarraInicial = 400f;
    public bool esCompleja = false;
    private void OnEnable()
    {
        if (data == null) { Debug.LogWarning("[ConfigSenal] data no asignado."); return; }
        data.tiempoLimite = tiempoLimite;
        data.anchoBarraInicial = anchoBarraInicial;
        data.esCompleja = esCompleja;
    }
}

```

Data

Clases de datos en tiempo de ejecucion (MonoBehaviours y ScriptableObjects) que almacenan el estado del jugador, la camara, los dialogos y cada minijuego. Actuan como la 'memoria' viva del juego.

Archivo	Líneas	Ruta relativa
CamaraData.cs	13	Data/Camara/CamaraData.cs
CuerpoUIData.cs	24	Data/Cuerpo/CuerpoUIData.cs
DialogoData.cs	13	Data/Dialogo/DialogoData.cs
DialogoUIData.cs	29	Data/Dialogo/DialogoUIData.cs
JugadorData.cs	40	Data/Jugador/JugadorData.cs
JugadorHabilidadesData.cs	49	Data/Jugador/JugadorHabilidadesData.cs
DistractorCerebro.cs	48	Data/Minijuego/Cabeza/DistractorCerebro.cs

MinijuegoCerebroData.cs	45	Data/Minijuego/Cabeza/MinijuegoCerebroData.cs
MinijuegoCantoData.cs	44	Data/Minijuego/Cantar/MinijuegoCantoData.cs
NotaCanto.cs	65	Data/Minijuego/Cantar/NotaCanto.cs
MinijuegoSaludoData.cs	38	Data/Minijuego/Mano/MinijuegoSaludoData.cs
MinijuegoSenalData.cs	38	Data/Minijuego/Mano/MinijuegoSenalData.cs
PanelManoData.cs	21	Data/Minijuego/Mano/PanelManoData.cs
CeldaCuadrículaData.cs	48	Data/Minijuego/Pierna/CeldaCuadrículaData.cs
MinijuegoPiernaData.cs	88	Data/Minijuego/Pierna/MinijuegoPiernaData.cs
MinijuegoResponderData.cs	61	Data/Minijuego/boca/MinijuegoResponderData.cs
PanelBocaData.cs	17	Data/Minijuego/boca/PanelBocaData.cs
MinijuegoPechoData.cs	47	Data/Minijuego/pecho/MinijuegoPechoData.cs
ParteCuerpoData.cs	19	Data/Partes del cuerpo/ParteCuerpoData.cs
TransicionZonaData.cs	47	Data/transicion/TransicionZonaData.cs

CamaraData.cs

13 líneas

Scripts/Data/Camara/CamaraData.cs

Descripción	Contenedor de datos para la cámara principal del juego. Expone el Transform del jugador objetivo, la velocidad de suavizado (Smooth Follow) y el flag seguirJugador que CamaraController consulta cada frame. Al desactivarse seguirJugador durante diálogos o minijuegos, la cámara se inmoviliza. Dependencias: CamaraController (lee los datos), JugadorData (origen del transform).
--------------------	---

```

/// <summary>
/// Contenedor de datos para la cámara principal.
/// CamaraController lee estas referencias cada frame en LateUpdate.
/// Desactivar seguirJugador inmoviliza la cámara durante diálogos o minijuegos.
/// </summary>
using UnityEngine;
public class CamaraData : MonoBehaviour
{
    [Header("Objetivo")]
    public Transform jugador;
    [Header("Configuración")]
    public float velocidadSmooth = 5f;
    [Header("Estado")]
    public bool seguirJugador = true;
}

```

CuerpoUIData.cs

24 líneas

Scripts/Data/Cuerpo/CuerpoUIData.cs

Descripción	Contenedor de referencias para el panel de selección de partes del cuerpo. Guarda el panel raíz y el arreglo de ParteCuerpoData con sus botones asociados. También define los colores de estado bloqueado/desbloqueado que CuerpoUIController aplica. Dependencias: ParteCuerpoData, CuerpoUIController.
--------------------	--

```
using UnityEngine;
/// <summary>
/// Datos de la UI del cuerpo.
/// Expone el panel y el array de partes con sus botones.
/// Sin lógica, solo referencias.
/// </summary>
public class CuerpoUIData : MonoBehaviour
{
    [Header("Panel principal")]
    [Tooltip("Panel que contiene todos los botones de partes del cuerpo")]
    public GameObject panelCuerpo;
    [Header("Partes del cuerpo")]
    [Tooltip("Arrastrar aquí los GameObjects que tengan el componente ParteCuerpoData")]
    public ParteCuerpoData[] partes;
    [Header("Colores de estado")]
    [Tooltip("Color del botón cuando la parte está desbloqueada")]
    public Color colorDesbloqueado = Color.white;
    [Tooltip("Color del botón cuando la parte está bloqueada")]
    public Color colorBloqueado = Color.gray;
}
```

DialogoData.cs

13 líneas

Scripts/Data/Dialogo/DialogoData.cs

Descripción	Clase serializable que representa una conversación completa como un arreglo de DialogoLinea. Al ser [Serializable] puede configurarse inline en el Inspector de cualquier MonoBehaviour. DialogoController la consume línea a línea. Dependencias: DialogoLinea.
--------------------	--

```
using UnityEngine;
/// <summary>
/// Contenedor de una conversación completa.
/// Clase serializable, se rellena inline en el Inspector.
/// Sin lógica, solo datos.
/// </summary>
[System.Serializable]
public class DialogoData
{
    [Tooltip("Lista de líneas de esta conversación en orden")]
    public DialogoLinea[] lineas;
}
```

DialogoUIData.cs

29 líneas

Scripts/Data/Dialogo/DialogoUIData.cs

Descripción	Centraliza todas las referencias UI del panel de dialogo: el panel raíz, el TextMeshPro de texto, las imágenes de personaje izquierda/derecha y la tecla para avanzar. DialogoController lee estos datos en cada frame para saber que actualizar. Dependencias: DialogoController, TMPro.
--------------------	---

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;
/// <summary>
/// Referencias a los elementos UI del panel de diálogo.
/// MonoBehaviour, sin lógica, solo datos.
/// </summary>
public class DialogoUIData : MonoBehaviour
{
    [Header("Panel")]
    [Tooltip("Panel raíz del diálogo, se activa y desactiva")]
    public GameObject panelDialogo;
```

```
[Header("Texto")]
[Tooltip("TextMeshPro donde se muestra el texto de cada línea")]
public TextMeshProUGUI textoDialogo;
[Header("Imágenes de personaje")]
[Tooltip("Imagen del lado izquierdo")]
public Image imagenIzquierda;
[Tooltip("Imagen del lado derecho")]
public Image imagenDerecha;
[Header("Configuración")]
[Tooltip("Tecla para avanzar al siguiente diálogo")]
public KeyCode teclaAvanzar = KeyCode.Space;
}
```

JugadorData.cs

40 líneas

Scripts/Data/Jugador/JugadorData.cs

Descripción	Fuente de verdad sobre el estado del jugador. Centraliza velocidad máxima, aceleración, desaceleración, dirección de input, el flag puedeMoverse (respetado por colisiones y gestores de misa), referencias de audio y nombres de animación. JugadorController lee y escribe estos valores cada frame. Dependencias: JugadorController, ColisionDialogo, GestorMisa.
--------------------	--

```
using UnityEngine;
/// <summary>
/// Contenedor de datos del jugador.
/// No contiene lógica, solo expone las variables
/// que JugadorController necesita para operar.
/// </summary>
public class JugadorData : MonoBehaviour
{
[Header("Movimiento")]
[Tooltip("Velocidad máxima en unidades/segundo")]
public float velocidadMaxima = 5f;
[Tooltip("Cuando tan rápido alcanza la velocidad máxima (unidades/s²)")]
public float aceleracion = 20f;
[Tooltip("Cuando tan rápido frena al soltar el input (unidades/s²)")]
public float desaceleracion = 30f;
[Header("Estado (solo lectura)")]
[Tooltip("Dirección de input detectada este frame")]
public Vector3 direccionInput = Vector3.zero;
[Header("Condición para moverse")]
[Tooltip("Impide movimiento en ciertas acciones o cuando se desee")]
public bool puedeMoverse = true;

[Header("Audio")]
public AudioSource audioCaminata;
public AudioClip clipCaminata;

[Header("Animación")]
public Animator animador;
public SpriteRenderer spriteRenderer;

[Header("Nombres de animación")]
public string animCaminando = "Caminando";
public string animQuieto = "Quieto";
}
```

JugadorHabilidadesData.cs

49 líneas

Scripts/Data/Jugador/JugadorHabilidadesData.cs

Descripción	Registra que partes del cuerpo tiene desbloqueadas el jugador mediante bools. Expone el método EstaDesbloqueada(ParteCuerpo) para que CuerpoUIController decida que botones habilitar. Al inicio todos los bools son true (todas las partes disponibles). Dependencias: CuerpoUIController, ParteCuerpo (enum).
--------------------	---

```
using UnityEngine;
/// <summary>
/// Fuente de verdad sobre que partes del cuerpo tiene desbloqueadas el jugador.
/// Solo datos, sin lógica.
```



```

posicionOriginal = GetComponent<RectTransform>().anchoredPosition;
}

/// <summary>
/// Resetea el distractor a su posicion original con nueva velocidad.
/// </summary>
public void Resetear(float nuevaVelocidad)
{
    GetComponent<RectTransform>().anchoredPosition = posicionOriginal;
    velocidadActual = nuevaVelocidad;
    activo = true;
    if (imagen != null)
    {
        imagen.gameObject.SetActive(true);
    }

    /// <summary>
    /// Vuelve a posicion original al ser clickeado.
    /// </summary>
    public void AlClickear(float velocidadMin, float velocidadMax)
    {
        Resetear(Random.Range(velocidadMin, velocidadMax));
    }
}

```

MinijuegoCerebroData.cs

45 lineas

Scripts/Data/Minijuego/Cabeza/MinijuegoCerebroData.cs

Descripcion	Contiene todas las referencias UI y parametros del minijuego de cabeza (cerebro): el panel, la imagen del cerebro, los cuatro distractores, y valores de perdida/ganancia de alpha, velocidad y radio de colision. MinijuegoCerebroController escribe el estado (tiempoTranscurrido, minijuegoActivo). Dependencias: DistractorCerebro, MinijuegoCerebroController.
--------------------	--

```
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Datos del minijuego de cerebro.
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoCerebroData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelCerebro;

    [Header("Cerebro")]
    [Tooltip("Image del cerebro central")]
    public Image imagenCerebro;
    [Tooltip("RectTransform del cerebro para detectar colision")]
    public RectTransform rectCerebro;

    [Header("Distractores")]
    [Tooltip("Los 4 distractores, cada uno con su Image y RectTransform")]
    public DistractorCerebro[] distractores;

    [Header("Configuracion cerebro")]
    [Tooltip("Cuanto alpha pierde por segundo")]
    public float perdidaAlpha = 0.08f;
    [Tooltip("Cuanto alpha gana al presionar espacio")]
    public float gananciaAlpha = 0.3f;

    [Header("Configuracion distractores")]
    public float velocidadMin = 60f;
    public float velocidadMax = 160f;
    [Tooltip("Radio de colision con el cerebro en pixeles")]
    public float radioColision = 50f;

    [Header("Configuracion de victoria")]
    [Tooltip("Segundos que debe sobrevivir para ganar")]
    public float tiempoSobrevivir = 15f;

    // ■■ Estado ████████████████████████████████████████████████████████
    [HideInInspector] public float tiempoTranscurrido = 0f;
    [HideInInspector] public bool minijuegoActivo = false;
}
```

MinijuegoCantoData.cs

44 lineas

Scripts/Data/Minijuego/Cantar/MinijuegoCantoData.cs

Descripcion	Almacena los pools de notas por carril (0-3), los botones de carril, y configuracion de velocidad, intervalo e intervalos de victoria. El estado (notasAcertadas, fallosActuales, timerNota) es escrito por MinijuegoCantoController. Dependencias: NotaCanto, MinijuegoCantoController.
--------------------	--

[illegible]

NotaCanto.cs

65 lineas

Scripts/Data/Minijuego/Cantar/NotaCanto.cs

Descripcion	Representa una nota musical individual del minijuego de canto. Vive en cada GameObject de nota dentro del pool. Activar() la posiciona y la pone en movimiento; Desactivar() la retorna al origen y oculta la imagen. MoverAbajo() la desplaza cada frame segun su velocidad. Dependencias: MinijuegoCantoController.
--------------------	---

[illegible]

```
[Tooltip("Posicion inicial de esta nota, asignar manualmente en el Inspector")]
public Vector2 posicionInicial;
private RectTransform _rect;
private void Start()
{
    _rect = GetComponent<RectTransform>();
}
public void Activar(int carrilAsignado, float velocidadAsignada)
{
    if (_rect == null)
        _rect = GetComponent<RectTransform>();
    carril = carrilAsignado;
    velocidad = velocidadAsignada;
    activa = true;
    _rect.anchoredPosition = posicionInicial;
    if (imagen != null)
        imagen.gameObject.SetActive(true);
}
public void Desactivar()
{
    activa = false;
    if (_rect == null)
        _rect = GetComponent<RectTransform>();
    _rect.anchoredPosition = posicionInicial;
    if (imagen != null)
        imagen.gameObject.SetActive(false);
}
public Vector2 PosicionActual => GetComponent<RectTransform>().anchoredPosition;
public void MoverAbajo(float delta)
{
    if (_rect == null)
        _rect = GetComponent<RectTransform>();
    _rect.anchoredPosition += Vector2.down * velocidad * delta;
}
}
```

MinijuegoSaludoData.cs

38 líneas

Scripts/Data/Minijuego/Mano/MinijuegoSaludoData.cs

Descripcion	Datos del minijuego de saludo. Contiene referencias a los dos botones (arriba/abajo), imagenes de las manos, objeto de exito, barra de tiempo y configuracion de tiempos. El flag botonCorrectoEsArriba se asigna aleatoriamente al iniciar. Dependencias: MinijuegoSaludoController.
--------------------	---

```
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos del minijuego de saludo de mano.
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoSaludoData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelSaludo;
    [Header("Botones")]
    public Button botonArriba;
    public Button botonAbajo;
    [Tooltip("Imagen de la mano del otro personaje (va en el boton correcto aleatorio)")]
    public Image imagenManoOtro;
    [Tooltip("Imagen de tu mano (va en el boton incorrecto)")]
    public Image imagenManoJugador;
    [Header("Imagen de exito")]
    public GameObject imagenExito;
    [Header("Barra de tiempo")]
    [Tooltip("RectTransform de la barra que se acorta con el tiempo")]
    public RectTransform barraTiempo;
```

```
[Tooltip("Ancho inicial de la barra en pixeles")]  
public float anchoBarraInicial = 400f;  
  
[Header("Configuracion")]  
public float tiempoLimite = 3f;  
public float tiempoExito = 1.2f;  
  
// ■■■ Estado ████████████████████████████████████████████████████████████████  
[HideInInspector] public bool botonCorrectoEsArriba;  
}
```

MinijuegoSenalData.cs

38 lineas

Scripts/Data/Minijuego/Mano/MinijuegoSenalData.cs

Descripcion	Datos del minijuego de senal de la cruz. Soporta modo simple (4 botones) y modo complejo (mas pasos) controlado por el flag esCompleja. Guarda el arreglo de botones para cada modo y el indice actual en la secuencia. Dependencias: MinijuegoSenalController, ConfigSenal.
--------------------	--

[illegible]

PanelManoData.cs

21 lineas

Scripts/Data/Minijuego/Mano/PanelManoData.cs

Descripcion	Datos del panel intermedio de la mano derecha. El jugador ve dos botones: Saludo o Senal. Al pulsar, PanelManoController dispara el evento correspondiente que MinijuegoManager escucha para lanzar el minijuego correcto. Dependencias: PanelManoController.
--------------------	---

```
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Datos del panel intermedio de la mano.
/// El jugador elige entre Saludo o Señal.
/// Solo datos, sin lógica.
/// </summary>
public class PanelManoData : MonoBehaviour
{
```

```
[Header("Panel")]
[Tooltip("Panel ra■z, se activa al seleccionar ManoDerecha")]
public GameObject panelMano;

[Header("Botones")]
[Tooltip("Bot■n para lanzar el minijuego de saludo")]
public Button botonSaludo;

[Tooltip("Bot■n para lanzar el minijuego de se■al")]
public Button botonSenal;
}
```

CeldaCuadrículaData.cs

48 lineas

Scripts/Data/Minijuego/Pierna/CeldaCuadrriculaData.cs

Descripcion	Representa una celda de la cuadrícula 3×3 del minijuego de pierna. Registra su posición (columna, fila), la parte ocupante, el botón UI y la imagen que muestra. Limpiar() resetea la celda al estado vacío. EstaOcupada es una propiedad calculada. Dependencias: MinijuegoPiernaController.
--------------------	---

MinijuegoPiernaData.cs

88 lineas

Scripts/Data/Minijuego/Pierna/MinijuegoPiernaData.cs

Descripcion	Datos del minijuego de ensamblaje de pierna. Referencia las 9 celdas, las imagenes del panel de orden (cadera→rodilla→pie), los sprites de cada parte y lineas de conexion visual. ObtenerSprite() y ObtenerImagenOrden() proveen acceso tipado. El orden aleatorio se genera al iniciar. Dependencias: CeldaCuadricolaData, MinijuegoPiernaController.
--------------------	---

[illegible]

```
case 1: return imagenOrden2;
case 2: return imagenOrden3;
default:
Debug.LogWarning($"[MinijuegoPiernaData] Indice de orden fuera de rango: {indice}");
return null;
}
}
#endregion
}
```

MinijuegoResponderData.cs

61 lineas

Scripts/Data/Minijuego/boca/MinijuegoResponderData.cs

Descripción	Datos del minijuego de preguntas. Contiene la imagen de pregunta, barra de tiempo, tres botones con textos configurables, sistema de ayuda con alpha progresivo y el índice de la respuesta correcta. ConfigResponder escribe el contenido de la pregunta antes de iniciar. Dependencias: MinijuegoResponderController, ConfigResponder.
--------------------	--

[illegible]

```
[HideInInspector] public float multiplicadorVelocidad = 1f;
[HideInInspector] public bool esperandoRespuesta = false;
}
```

PanelBocaData.cs

17 lineas

Scripts/Data/Minijuego/boca/PanelBocaData.cs

Descripcion	Datos del panel intermedio de la boca. Ofrece al jugador elegir entre Responder (preguntas) o Cantar (notas musicales). PanelBocaController gestiona los eventos de seleccion. Dependencias: PanelBocaController.
--------------------	---

```
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos del panel intermedio de la boca.
/// El jugador elige entre Responder o Cantar.
/// Solo datos, sin logica.
/// </summary>
public class PanelBocaData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelBoca;
    [Header("Botones")]
    public Button botonResponder;
    public Button botonCantar;
}
```

MinijuegoPechoData.cs

47 lineas

Scripts/Data/Minijuego/pecho/MinijuegoPechoData.cs

Descripcion	Datos del minijuego de ritmo del pecho. El jugador debe presionar el boton del corazon cuando el circulo externo decreciente coincida con el corazon fijo. Configura rondas totales, tamanos y velocidades. El estado de ronda y tamaño actual es escrito por MinijuegoPechoController. Dependencias: MinijuegoPechoController, ConfigPecho.
--------------------	--

```
using UnityEngine;
using UnityEngine.UI;
/// <summary>
/// Datos del minijuego de ritmo del pecho.
/// Solo datos, sin logica.
/// </summary>
public class MinijuegoPechoData : MonoBehaviour
{
    [Header("Panel")]
    public GameObject panelPecho;
    [Header("Visual")]
    [Tooltip("RectTransform del circulo externo que se achica")]
    public RectTransform circuloExterno;
    [Tooltip("RectTransform del corazon fijo en el centro")]
    public RectTransform corazon;
    [Tooltip("Boton del corazon que el jugador presiona")]
    public Button botonCorazon;
    [Header("Configuracion de rondas")]
    [Tooltip("Cuantas veces debe acertar para ganar")]
    public int rondasTotales = 4;
    [Tooltip("Tamano inicial del circulo externo")]
    public float tamanoInicial = 400f;
    [Tooltip("Tamano del corazon fijo (referencia para comparar)")]
    public float tamanoCorazon = 100f;
    [Tooltip("Margen de tolerancia para considerar acierto")]
    public float margenAcierto = 30f;
    [Tooltip("Velocidad minima de achicamiento")]
    public float velocidadMin = 80f;
    [Tooltip("Velocidad maxima de achicamiento")]
}
```


[illegible]

ParteCuerpoData.cs

19 lineas

Scripts/Data/Partes del cuerpo/ParteCuerpoData.cs

Descripcion	Asocia un valor del enum ParteCuerpo con su boton UI e imagen. CuerpoUIController itera el arreglo de ParteCuerpoData para registrar listeners y aplicar colores segun el estado de JugadorHabilidadesData. Dependencias: CuerpoUIController, ParteCuerpo.
--------------------	--

```
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Datos de una parte del cuerpo vinculada a un bot n de la UI.
/// MonoBehaviour para poder asignarse como componente en el Inspector.
/// Solo datos, sin l gica.
/// </summary>

public class ParteCuerpoData : MonoBehaviour
{
    [Tooltip("Qu  parte del cuerpo representa este componente")]
    public ParteCuerpo parte;

    [Tooltip("Bot n de la UI asociado a esta parte")]
    public Button boton;

    [Tooltip("Imagen del bot n para feedback visual de bloqueo/desbloqueo")]
    public Image imagenBoton;
}
```

TransicionZonaData.cs

47 lineas

Scripts/Data/transicion/TransicionZonaData.cs

Descripcion	Datos de la transicion de zona con cuadro negro. Define el cuadro de fade, velocidad de transicion, zonas a activar/desactivar, puntos de destino para jugador y camara, y audio de transicion. El flag SeguirJugadorSiguiente indica si la camara debe retomar el follow tras la transicion. Dependencias: TransicionZonaController.
--------------------	---

```
using UnityEngine;

/// <summary>
/// Datos de la transición de zona.
/// Sin lógica, solo referencias y configuraciones.
/// </summary>
public class TransicionZonaData : MonoBehaviour
{
    [Header("Cuadro negro")]
    [Tooltip("Transform del cuadro negro hijo de la cámara")]
    public Transform cuadroNegro;

    [Tooltip("Velocidad a la que sube y baja el cuadro negro")]
    public float velocidadTransicion = 2f;

    [Header("Zonas")]
    [Tooltip("GameObject de la nueva zona a activar")]
    public GameObject zonaEntrada;

    [Tooltip("GameObject de la zona anterior a desactivar")]
    public GameObject zonaSalida;

    [Header("Puntos de destino")]
    [Tooltip("Punto donde se teletransporta el jugador")]
    public Transform puntoJugador;

    [Tooltip("Punto donde se teletransporta la cámara")]
    public Transform puntoCamara;

    [Header("Referencias")]
    [Tooltip("Transform de la cámara principal")]
    public Transform camara;
}
```

```
[Header("Audio")]
[Tooltip("AudioSource que reproduce el sonido de transición")]
public AudioSource audioTransicion;
[Tooltip("Nuevo AudioClip de caminata para la zona destino")]
public AudioClip nuevaCaminata;

[Header("Seguir Al Jugador Luego")]
public bool SeguirJugadorSiguiente;
}
```

Enumeraciones

Enumeraciones de C# que definen conjuntos de valores con nombre. Facilitan la lectura del código y evitan el uso de enteros mágicos al representar partes del cuerpo, tipos de acción, etc.

Archivo	Líneas	Ruta relativa
TipoAccion.cs	12	Enums/Acciones Mentales/TipoAccion.cs
PartePierna.cs	9	Enums/Minijuego/Pierna/PartePierna.cs
ParteCuerpo.cs	12	Enums/Partes del cuerpo/ParteCuerpo.cs

TipoAccion.cs

12 líneas

Scripts/Enums/Acciones Mentales/TipoAccion.cs

Descripción	Enumeración de los cinco tipos de acción que puede devolver un minijuego: Postura, Mano, Pecho, Cabeza, Boca. MisionBase usa TipoAccion junto con ValorEsperado para evaluar si el resultado reportado coincide con la misión activa. Extender este enum al agregar nuevos minijuegos.
--------------------	--

```
/// <summary>
/// Tipos de acción que un minijuego puede devolver.
/// Agregar nuevos tipos aquí según se expanda el juego.
/// </summary>
public enum TipoAccion
{
    Postura,
    Mano,
    Pecho,
    Cabeza,
    Boca
}
```

PartePierna.cs

9 líneas

Scripts/Enums/Minijuego/Pierna/PartePierna.cs

Descripción	Define las tres partes ensamblables de la pierna: Cadera, Rodilla, Pie. El minijuego de pierna genera un orden aleatorio con estos valores y CeldaCuadrículaData los usa como identificador de que parte ocupa cada celda.
--------------------	--

```
/// <summary>
/// Partes que componen la pierna en el minijuego.
/// </summary>
public enum PartePierna
{
    Cadera,
    Rodilla,
    Pie
}
```

ParteCuerpo.cs

12 líneas

Scripts/Enums/Partes del cuerpo/ParteCuerpo.cs

Descripción	Enumeración de las partes del cuerpo disponibles: Piernas, Pecho, Cabeza, Boca, ManoDerecha. CuerpoUIController dispara OnParteSeleccionada con este valor, que MinijuegoManager interpreta para lanzar el minijuego correcto.
--------------------	--

```

/// <summary>
/// Partes del cuerpo disponibles en el juego.
/// Agregar nuevas partes aquí según se expanda el juego.
/// </summary>
public enum ParteCuerpo
{
    Piernas,
    Pecho,
    Cabeza,
    Boca,
    ManoDerecha
}

```

Estructuras

Structs de C# que agrupan datos relacionados de forma ligera y eficiente. Se usan para representar resultados de acciones y líneas de diálogo sin la sobrecarga de un MonoBehaviour.

Archivo	Líneas	Ruta relativa
AccionResultado.cs	15	Structuras/Acciones Mentales/AccionResultado.cs
DialogoLinea.cs	19	Structuras/Dialogo/DialogoLinea.cs

AccionResultado.cs

15 líneas

Scripts/Structuras/Acciones Mentales/AccionResultado.cs

Descripción	Struct ligero que encapsula el resultado de un minijuego: un TipoAccion y un entero valor. MisionesGlobal.ReportarResultado() recibe este struct y lo compara con la misión activa. Al ser struct evita allocations en el heap y se pasa por valor. Dependencias: TipoAccion.
--------------------	---

```

/// <summary>
/// Resultado que devuelve cualquier minijuego al terminar.
/// Solo datos, sin lógica.
/// </summary>
public struct AccionResultado
{
    public TipoAccion tipo;
    public int valor;
    public AccionResultado(TipoAccion tipo, int valor)
    {
        this.tipo = tipo;
        this.valor = valor;
    }
}

```

DialogoLinea.cs

19 líneas

Scripts/Structuras/Dialogo/DialogoLinea.cs

Descripción	Struct serializable que representa una línea individual de diálogo: el texto, el sprite del personaje izquierdo y el sprite del personaje derecho. DialogoData contiene un arreglo de DialogoLinea; DialogoController los itera para actualizar la UI. Dependencias: DialogoData, DialogoController.
--------------------	--

```

using UnityEngine;

/// <summary>
/// Datos de una línea de diálogo.
/// Struct puro, sin lógica.
/// </summary>

```

```
[System.Serializable]
public struct DialogoLinea
{
    [Tooltip("Texto de esta línea de diálogo")]
    [TextArea]
    public string texto;
    [Tooltip("Imagen del personaje que habla")]
    public Sprite imagen;
    [Tooltip("Si es true la imagen aparece a la izquierda, si es false a la derecha")]
    public bool esIzquierda;
}
```

Interfaces

Contratos de interfaz que definen el comportamiento minimo que deben implementar ciertas clases. Permiten el polimorfismo y la extension del sistema de misiones sin romper el codigo existente.

Archivo	Lineas	Ruta relativa
MisionBase.cs	43	Interfaces/Mision/MisionBase.cs

MisionBase.cs

43 lineas

Scripts/Interfaces/Mision/MisionBase.cs

Descripción	Clase base abstracta para todas las misiones del juego. No es MonoBehaviour, se instancia como objeto puro y se pasa a MisionesGlobal. Define TipoEsperado y ValorEsperado, el metodo EvaluarResultado() que compara el AccionResultado recibido, y el metodo abstracto Actuar() que cada mision concreta implementa. Dependencias: TipoAccion, AccionResultado.
--------------------	--

[illegible]

```
#endregion
}
```

Managers

Orquestadores centrales del juego. Escuchan eventos del sistema y delegan la logica al componente apropiado. El MinijuegoManager coordina todos los minijuegos; el MisionesGlobal gestiona el progreso de misiones.

Archivo	Lineas	Ruta relativa
MinijuegoManager.cs	115	Managers/Minijuegos/MinijuegoManager.cs
MisionesGlobal.cs	114	Managers/Misiones/MisionesGlobal.cs

MinijuegoManager.cs

115 lineas

Scripts/Managers/Minijuegos/MinijuegoManager.cs

Descripcion	Orquestador central de minijuegos. Se suscribe a <code>CuerpoUIController.OnParteSeleccionada</code> y a los eventos de sub-seleccion de <code>PanelManoController</code> y <code>PanelBocaController</code> . Segun la parte elegida, llama a <code>Iniciar()</code> en el controlador de minijuego correspondiente. Mantiene bajo acoplamiento: solo conoce las referencias, no la logica interna. Dependencias: todos los controladores de minijuego, <code>CuerpoUIController</code> .
--------------------	--

```

private void AlSeleccionarParte(ParteCuerpo parte)
{
switch (parte)
{
case ParteCuerpo.Piernas:
if (minijuegoPierna == null) { Advertir(parte); return; }
minijuegoPierna.Iniciar();
break;
case ParteCuerpo.Pecho:
if (minijuegoPecho == null) { Advertir(parte); return; }
minijuegoPecho.Iniciar();
break;
case ParteCuerpo.Cabeza:
if (minijuegoCerebro == null) { Advertir(parte); return; }
minijuegoCerebro.Iniciar();
break;
case ParteCuerpo.ManoDerecha:
if (panelMano == null) { Advertir(parte); return; }
panelMano.AbrirPanel();
break;
case ParteCuerpo.Boca:
if (panelBoca == null) { Advertir(parte); return; }
panelBoca.AbrirPanel();
break;
default:
Debug.LogWarning($"[MinijuegoManager] No hay minijuego implementado para: {parte}");
break;
}
}

private void AlSeleccionarSaludo()
{
if (minijuegoSaludo == null) { Debug.LogWarning("[MinijuegoManager] minijuegoSaludo no asignado.");
return; ...
minijuegoSaludo.Iniciar();
}

private void AlSeleccionarSenal()
{
if (minijuegoSenal == null) { Debug.LogWarning("[MinijuegoManager] minijuegoSenal no asignado."); return; }
minijuegoSenal.Iniciar();
}

private void AlSeleccionarResponder()
{
if (minijuegoResponder == null) { Debug.LogWarning("[MinijuegoManager] minijuegoResponder no asignado.");
r ...
minijuegoResponder.Iniciar();
}

private void AlSeleccionarCantar()
{
if (minijuegoCanto == null) { Debug.LogWarning("[MinijuegoManager] minijuegoCanto no asignado."); return; }
minijuegoCanto.Iniciar();
}

private void Advertir(ParteCuerpo parte)
{
Debug.LogWarning($"[MinijuegoManager] El minijuego de {parte} no esta asignado en el Inspector.");
}

#endregion
}

```

MisionesGlobal.cs

114 líneas

Scripts/Managers/Misiones/MisionesGlobal.cs

Descripcion

Singleton persistente (DontDestroyOnLoad) que mantiene una mision activa a la vez. AsignarMision() registra la nueva mision; ReportarResultado() la evalua y, si acierta, llama Actuar(); si falla, llama ActuarFallo() en MisionPosicion. CancelarMision() limpia sin ejecutar. Previene duplicados de singleton. Dependencias: MisionBase, AccionResultado, MisionPosicion.


```

    misionResuelta.Actuar();
}
else
{
    Debug.Log($"[MisionesGlobal] Fallo. Esperaba tipo={misionResuelta.TipoEsperado} " +
        $"valor={misionResuelta.ValorEsperado}. " +
        $"Recibido tipo={resultado.tipo} valor={resultado.valor}.");
    // Si la mision soporta fallo lo notifica
    if (misionResuelta.is MisionPosicion misionPosicion)
    misionPosicion.ActuarFallo();
}
}

/// <summary>
/// Cancela la mision activa sin ejecutarla.
/// </summary>
public void CancelarMision()
{
    if (_misionActiva == null)
    {
        Debug.Log("[MisionesGlobal] No hay mision activa que cancelar.");
        return;
    }
    Debug.Log("[MisionesGlobal] Mision cancelada.");
    _misionActiva = null;
}
#endregion
}

```

Misa

Scripts específicos de la escena de Misa. Controlan el flujo narrativo de la ceremonia, la barra de vida/energía del jugador y las transiciones entre etapas de la escena.

Archivo	Líneas	Ruta relativa
EscenaMisa.cs	44	Misa/EscenaMisa.cs
GestorMisa.cs	302	Misa/GestorMisa.cs
VidaUI.cs	34	Misa/VidaUI.cs

EscenaMisa.cs

44 líneas

Scripts/Misa/EscenaMisa.cs

Descripción	Clase serializable que describe una escena individual dentro de la secuencia de misa: nombre, si muestra título, diálogos de inicio/éxito/fallo, tipo de acción y valor esperado para la misión, música opcional, animación del sacerdote y su punto de destino, objetos a activar/desactivar, y visuales del jugador. GestorMisa itera un arreglo de EscenaMisa configurado en el Inspector. Dependencias: GestorMisa, DialogoData.
--------------------	--

```

using UnityEngine;

/// <summary>
/// Datos de una escena individual de la misa.
/// Serializable para configurar inline en el Inspector del GestorMisa.
/// </summary>
[System.Serializable]
public class EscenaMisa
{
    [Header("Identificación")]
    public string nombreEscena;
    [Tooltip("Si es true muestra el título grande antes del diálogo inicial")]
    public bool mostrarTitulo;
    public string textoTitulo;
}

```



```
[Header("Di■Logos")]
public DialogoData dialogoInicial;
public DialogoData dialogoExito;
public DialogoData dialogoFallo;

[Header("Misi■n")]
public TipoAccion tipoAccion;
public int valorEsperado;

[Header("M■sica")]
[Tooltip("Si no es null cambia la m■sica al iniciar esta escena")]
public AudioClip musicaOpcional;

[Header("Sacerdote")]
public string animacionSacerdote;
[Tooltip("Si no es null mueve al sacerdote a este punto")]
public Transform puntoDestino;
public float velocidadSacerdote = 2f;

[Header("Objetos")]
[Tooltip("Se activan al iniciar la escena")]
public GameObject[] activar;
[Tooltip("Se desactivan al iniciar la escena")]
public GameObject[] desactivar;

[Header("Sprites del jugador en escena")]
public GameObject[] visualesJugador;
}
```

GestorMisa.cs

302 lineas

Scripts/Misa/GestorMisa.cs

Descripcion	Controlador principal de la ceremonia de misa. Al entrar el jugador al area trigger y presionar espacio, inicia la coroutine IniciarFlujo que ejecuta cada EscenaMisa en secuencia. Por cada escena: configura objetos, mueve al sacerdote, muestra titulo, reproduce dialogo inicial, espera resultado de mision, y segun exito/fallo avanza o reinicia. Al completar todas las escenas activa el objeto de comunion (ganar). Dependencias: EscenaMisa, DialogoController, MisionesGlobal, MisionPosicion, VidaUI.
--------------------	---

[illegible]


```
if (escena.puntoDestino != null && sacerdote != null)
yield return StartCoroutine(MoverSacerdote(escena));
if (escena.mostrarTitulo)
yield return StartCoroutine(MostrarTitulo(escena.textoTitulo));
yield return StartCoroutine(EsperarDialogo(escena.dialogoInicial));
yield return StartCoroutine(EsperarMision(escena));
}

private IEnumerator EsperarMision(EscenaMisa escena)
{
    if (escena.visualesJugador != null)
        foreach (GameObject v in escena.visualesJugador)
            if (v != null) v.SetActive(true);
    if (pensamientoJugador != null)
        pensamientoJugador.SetActive(true);
    bool resultado = false;
    bool recibido = false;
    MisionPosicion mision = new MisionPosicion(
        escena.tipoAccion,
        escena.valorEsperado,
        () => { resultado = true; recibido = true; },
        () => { resultado = false; recibido = true; }
    );
    if (MisionesGlobal.Instancia != null)
        MisionesGlobal.Instancia.AsignarMision(mision);
    yield return new WaitUntil(() => recibido);
    if (pensamientoJugador != null)
        pensamientoJugador.SetActive(false);
    if (escena.visualesJugador != null)
        foreach (GameObject v in escena.visualesJugador)
            if (v != null) v.SetActive(false);
    if (resultado)
        yield return StartCoroutine(ALAcertar(escena));
    else
        yield return StartCoroutine(ALFallar(escena));
}

private IEnumerator ALAcertar(EscenaMisa escena)
{
    yield return StartCoroutine(EsperarDialogo(escena.dialogoExito));
    _escenaActual++;
    yield return StartCoroutine(EjecutarEscena(_escenaActual));
}

private IEnumerator ALFallar(EscenaMisa escena)
{
    yield return StartCoroutine(EsperarDialogo(escena.dialogoFallo));
    // Bajar cortina, resetear jugador, subir cortina y reintentar
    yield return StartCoroutine(MoverCuadro(0f));
    if (transformJugador != null && puntoResetJugador != null)
        transformJugador.position = puntoResetJugador.position;
    yield return new WaitForSeconds(0.3f);
    yield return StartCoroutine(MoverCuadro(2f));
    yield return StartCoroutine(EjecutarEscena(_escenaActual));
}

#endregion
// ████████████████████████████████████████████████████████████████████████████

#region Ganar / Reiniciar
private IEnumerator Ganar()
{
    if (objetoGanar != null) objetoGanar.SetActive(true);
    if (vidaUI != null) vidaUI.gameObject.SetActive(false);
    if (jugadorData != null) jugadorData.puedeMoverse = true;
    if (visualJugadorReal != null) visualJugadorReal.SetActive(true);
    _enFlujo = false;
    yield break;
}

private IEnumerator Reiniciar()
{
```

```
yield return StartCoroutine(MoverCuadro(0f));
_escenaActual = 0;
vidaUI?.Resetear();
if (transformJugador != null && puntoResetJugador != null)
    transformJugador.position = puntoResetJugador.position;
yield return new WaitForSeconds(0.5f);
yield return StartCoroutine(MoverCuadro(2f));
// Volver a ejecutar desde la escena 0 directamente
yield return StartCoroutine(EjecutarEscena(_escenaActual));
}

#endregion

// ████████████████████████████████████████████████████████████████████████████

#region Helpers
private IEnumerator EsperarDialogo(DialogoData dialogo)
{
    if (dialogo == null) yield break;
    bool terminado = false;
    System.Action handler = () => terminado = true;
    DialogoController.OnDialogoTerminado += handler;
    _dialogoController.Iniciar(dialogo);
    yield return new WaitUntil(() => terminado);
    DialogoController.OnDialogoTerminado -= handler;
}

private IEnumerator MostrarTitulo(string texto)
{
    if (panelTitulo == null) yield break;
    if (textoTitulo != null) textoTitulo.text = texto;
    panelTitulo.SetActive(true);
    yield return new WaitForSeconds(2.5f);
    panelTitulo.SetActive(false);
}

private void ConfigurarObjetos(EscenaMisa escena)
{
    if (escena.activar != null)
        foreach (GameObject obj in escena.activar)
            if (obj != null) obj.SetActive(true);
    if (escena.desactivar != null)
        foreach (GameObject obj in escena.desactivar)
            if (obj != null) obj.SetActive(false);
}

private IEnumerator MoverSacerdote(EscenaMisa escena)
{
    while (Vector3.Distance(sacerdote.position, escena.puntoDestino.position) > 0.05f)
    {
        sacerdote.position = Vector3.MoveTowards(
            sacerdote.position,
            escena.puntoDestino.position,
            escena.velocidadSacerdote * Time.deltaTime
        );
        yield return null;
    }
    sacerdote.position = escena.puntoDestino.position;
}

private IEnumerator MoverCuadro(float yDestino)
{
    if (cuadroNegro == null) yield break;
    Vector3 posInicial = cuadroNegro.localPosition;
    Vector3 posDestino = new Vector3(posInicial.x, yDestino, posInicial.z);
    float distancia = Mathf.Abs(posInicial.y - yDestino);
    if (distancia < 0.001f) yield break;
    float tiempoTotal = distancia / velocidadCuadro;
    float tiempoTranscurrido = 0f;
    while (tiempoTranscurrido < tiempoTotal)
    {
        tiempoTranscurrido += Time.deltaTime;
        float t = Mathf.Clamp01(tiempoTranscurrido / tiempoTotal);
        cuadroNegro.localPosition = Vector3.Lerp(posInicial, posDestino, t);
    }
}
```

```

yield return null;
}
cuadroNegro.localPosition = posDestino;
}
#endregion
}

```

VidaUI.cs

34 líneas

Scripts/Misa/VidaUI.cs

Descripción	Muestra y gestiona los corazones de vida del jugador durante la misa. Resetear() activa los N corazones al inicio o al reiniciar. PerderVida() desactiva el ultimo corazon activo. SinVidas es una propiedad que GestorMisa puede consultar para decidir si reiniciar completamente. Dependencias: GestorMisa.
--------------------	--

```

using UnityEngine;
/// <summary>
/// Muestra los 3 corazones de vida del jugador.
/// Se desactiva uno por cada fallo.
/// </summary>
public class VidaUI : MonoBehaviour
{
    [Tooltip("Los 3 corazones en orden, el último es el primero en perderse")]
    public GameObject[] corazones;
    private int _vidasActuales;
    private void Start()
    {
        Resetear();
    }
    public void Resetear()
    {
        _vidasActuales = corazones.Length;
        foreach (GameObject c in corazones)
        {
            if (c != null) c.SetActive(true);
        }
    }
    public void PerderVida()
    {
        if (_vidasActuales <= 0) return;
        _vidasActuales--;
        corazones[_vidasActuales].SetActive(false);
    }
    public bool SinVidas => _vidasActuales <= 0;
}

```

Misiones

Implementaciones concretas del sistema de misiones. Cada clase extiende la base de mision y define las condiciones de inicio, completado y fallo de una tarea específica dentro del juego.

Archivo	Líneas	Ruta relativa
Mision01.cs	137	Misiones/Mision01.cs
MisionMinijuego.cs	238	Misiones/MisionMinijuego.cs
MisionPosicion.cs	36	Misiones/MisionPosicion.cs
MisionPosturaParado. cs	259	Misiones/MisionPosturaParado.cs

Mision01.cs

137 líneas

Scripts/Misiones/Mision01.cs

Descripcion	Primera mision del juego: introduce al jugador al sistema de misiones. Hereda de MisionBase y define las condiciones de completado para la accion inicial (tipicamente postura parado). Su metodo Actuar() desencadena el siguiente paso narrativo, como activar una zona o iniciar un dialogo. Dependencias: MisionBase, MisionesGlobal.
--------------------	---

```
using UnityEngine,
/// <summary>
/// Primera mision del juego.
/// Flujo: dialogo inicial → suscribe mision + activa PENSAMIENTOJUGADOR
/// → si falla: dialogo fallo → vuelve a suscribir + activar
/// → si acierta: dialogo exito → resto de logica a cargo del disenador
/// </summary>
```

```
public class Misión01 : MonoBehaviour
{
    [Header("Dialogos")]
    public DialogoData dialogoInicial;
    public DialogoData dialogoFallo;
    public DialogoData dialogoExito;

    [Header("Referencias")]
    public GameObject pensamientoJugador;

    [Header("Logicas de activacion")]
    public JugadorController controladorJugador;
    public AudioSource jugadorAudioSource;
    public GameObject[] VisualesJugador;
    public GameObject[] VisualesJugadorDesactivadas;
    public GameObject BurbujaEspaciadora;
    private DialogoController _dialogoController;

    //
```

```
#region Unity Callbacks
private void Start()
{
    _dialogoController = FindFirstObjectByType<DialogoController>();
    if (_dialogoController == null)
    {
        Debug.LogError("[Mision01] No se encontro DialogoController en escena.");
        return;
    }
    DesactivarPensamiento();
    IniciarDialogoInicial();
}
```

```
#endregion  
// ████████████████████████████████████████████████████████████████████████████  
  
#region Flujo de Mision  
  
private void IniciarDialogoInicial()  
{  
    DialogoController.OnDialogoTerminado += AlTerminarDialogoInicial;  
    _dialogoController.Iniciar(dialogoInicial);  
}  
  
private void AlTerminarDialogoInicial()  
{  
    DialogoController.OnDialogoTerminado -= AlTerminarDialogoInicial;  
    SuscribirseMision();  
}  
  
private void SuscribirseMision()  
{  
    MisionPosicion mision = new MisionPosicion(  
        TipoAccion.Postura, 1,  
        AlAcertar, AlFallar  
    );  
    if (MisionesGlobal.Instancia == null)  
    {  
        Debug.LogError("[Mision01] MisionesGlobal no encontrado.");  
        return;  
    }  
    MisionesGlobal.Instancia.AsignarMision(mision);  
    ActivarPensamiento();
```

```
if (BurbujaEspaciadora != null)
    BurbujaEspaciadora.SetActive(true);
}

private void AlFallar()
{
    DesactivarPensamiento();
    DialogoController.OnDialogoTerminado += AlTerminarDialogoFallo;
    _dialogoController.Iniciar(dialogoFallo);
}

private void AlTerminarDialogoFallo()
{
    DialogoController.OnDialogoTerminado -= AlTerminarDialogoFallo;
    SuscribirseMision();
}

private void AlAcertar()
{
    DesactivarPensamiento();
    DialogoController.OnDialogoTerminado += AlTerminarDialogoExito;
    _dialogoController.Iniciar(dialogoExito);
}

private void AlTerminarDialogoExito()
{
    DialogoController.OnDialogoTerminado -= AlTerminarDialogoExito;
    if (controladorJugador != null)
        controladorJugador.enabled = true;
    foreach (var item in VisualesJugador)
        if (item != null) item.SetActive(true);
    foreach (var item in VisualesJugadorDesactivadas)
        if (item != null) item.SetActive(false);
    if (jugadorAudioSource != null)
        jugadorAudioSource.enabled = true;
}

#endregion

// ████████████████████████████████████████████████████████████████████████████████████

#region Helpers

private void ActivarPensamiento()
{
    if (pensamientoJugador != null)
        pensamientoJugador.SetActive(true);
}

private void DesactivarPensamiento()
{
    if (pensamientoJugador != null)
        pensamientoJugador.SetActive(false);
}

#endregion
```

MisionMinijuego.cs

238 lines

Scripts/Misiones/MisionMinijuego.cs

Descripcion	Mision que requiere completar un minijuego especifico. Hereda de MisionBase, abre el panel CuerpoUI para que el jugador seleccione la parte correcta del cuerpo y espera el resultado a traves de MisionesGlobal. Implementa Actuar() para avanzar el flujo narrativo tras el exito. Dependencias: MisionBase, CuerpoUIController, MisionesGlobal.
--------------------	--

```
using UnityEngine;
using System.Collections;
using System.Collections.Generic;

/// <summary>
/// Mision generica reutilizable.
/// Configura TipoAccion y respuestaEsperada desde el Inspector.
/// </summary>
public class MisionMinijuego : MonoBehaviour
{
```

[illegible]


```

DialogoController.OnDialogoTerminado += AlTerminarDialogoInicial;
_dialogoController.Iniciar(dialogoInicial);
}
private void AlTerminarDialogoInicial()
{
DialogoController.OnDialogoTerminado -= AlTerminarDialogoInicial;
if (habilidadesJugador != null)
{
DesbloquearHabilidad();
if (uiNuevaHabilidad != null) uiNuevaHabilidad.SetActive(true);
}
SuscribirMision();
}
private void DesbloquearHabilidad()
{
switch (habilidadADesbloquear)
{
case ParteCuerpo.Piernas: habilidadesJugador.tienePiernas = true; break;
case ParteCuerpo.Pecho: habilidadesJugador.tienePecho = true; break;
case ParteCuerpo.Cabeza: habilidadesJugador.tieneCabeza = true; break;
case ParteCuerpo.Boca: habilidadesJugador.tieneBoca = true; break;
case ParteCuerpo.ManoDerecha: habilidadesJugador.tieneManoDerecha = true; break;
}
}
private void SuscribirMision()
{
MisionPosicion mision = new MisionPosicion(
tipoAccion, respuestaEsperada,
AlAcertar, AlFallar
);
if (MisionesGlobal.Instancia != null)
MisionesGlobal.Instancia.AsignarMision(mision);
else
Debug.LogError("[MisionMini juego] MisionesGlobal no encontrado.");
if (pensamientoJugador != null)
pensamientoJugador.SetActive(true);
}
private void AlFallar()
{
if (pensamientoJugador != null) pensamientoJugador.SetActive(false);
DialogoController.OnDialogoTerminado += AlTerminarDialogoFallo;
_dialogoController.Iniciar(dialogoFallo);
}
private void AlTerminarDialogoFallo()
{
DialogoController.OnDialogoTerminado -= AlTerminarDialogoFallo;
if (_jugadorData != null) _jugadorData.puedeMoverse = true;
StartCoroutine(ReactivarSiguienteFrame());
}
private IEnumerator ReactivarSiguienteFrame()
{
yield return null;
yield return null;
_misionActiva = false;
}
private void AlAcertar()
{
if (pensamientoJugador != null) pensamientoJugador.SetActive(false);
DialogoController.OnDialogoTerminado += AlTerminarDialogoExito;
_dialogoController.Iniciar(dialogoExito);
}
private void AlTerminarDialogoExito()
{
DialogoController.OnDialogoTerminado -= AlTerminarDialogoExito;
StartCoroutine(Cinematica());
}
}
#endregion

```



```

/// </summary>
public class MisiónPosición : MisiónBase
{
    private readonly Action _onAcierto;
    private readonly Action _onFallo;

    public MisiónPosición(TipoAcción tipo, int valor, Action onAcierto, Action onFallo)
    : base(tipo, valor)
    {
        _onAcierto = onAcierto;
        _onFallo = onFallo;
    }

    /// <summary>
    /// Se llama cuando el resultado coincide con lo esperado.
    /// </summary>
    public override void Actuar()
    {
        _onAcierto?.Invoke();
    }

    /// <summary>
    /// Se llama cuando el resultado NO coincide con lo esperado.
    /// </summary>
    public void ActuarFallo()
    {
        _onFallo?.Invoke();
    }
}

```

MisionPosturaParado.cs

259 lineas

[Scripts/Misiones/MisionPosturaParado.cs](#)

Descripcion	Mision compleja de postura en posicion parado. Abre el panel CuerpoUI, espera que el jugador interactue con multiples partes del cuerpo en secuencia y valida que todas esten en la postura correcta. Al completarse desbloquea la siguiente fase del juego. Dependencias: MisionBase, CuerpoUIController, JugadorHabilidadesData, MisionesGlobal.
--------------------	--


```

_jugadorData.puedeMoverse = true;
// 9 - Desactivar este script
this.enabled = false;
}

/// <summary>
/// primer swap: desactiva A y activa B.
/// segundo swap: activa A y desactiva B.
/// </summary>
private void SwapSprites(bool primerSwap)
{
    foreach (SpriteRenderer sr in spritesDesactivar)
    if (sr != null) sr.enabled = !primerSwap;
    foreach (SpriteRenderer sr in spritesActivar)
    if (sr != null) sr.enabled = primerSwap;
}

private IEnumerator MoverCuadro(float yDestino)
{
    if (cuadroNegro == null) yield break;
    Vector3 posInicial = cuadroNegro.localPosition;
    Vector3 posDestino = new Vector3(posInicial.x, yDestino, posInicial.z);
    float distancia = Mathf.Abs(posInicial.y - yDestino);
    if (distancia < 0.001f) yield break;
    float tiempoTotal = distancia / velocidadCuadro;
    float tiempoTranscurrido = 0f;
    while (tiempoTranscurrido < tiempoTotal)
    {
        tiempoTranscurrido += Time.deltaTime;
        float t = Mathf.Clamp01(tiempoTranscurrido / tiempoTotal);
        cuadroNegro.localPosition = Vector3.Lerp(posInicial, posDestino, t);
        yield return null;
    }
    cuadroNegro.localPosition = posDestino;
}

#endregion
}

```

Modelos

Controladores MVC del juego. Implementan la lógica de presentación y comportamiento de cada entidad: jugador, cámara, diálogos, UI del cuerpo y todos los minijuegos. Son el núcleo de la jugabilidad.

Archivo	Líneas	Ruta relativa
CamaraController.cs	20	Modelos/Camara/CamaraController.cs
CuerpoUIController.cs	151	Modelos/Cuerpo/CuerpoUIController.cs
DialogoController.cs	160	Modelos/Dialogo/DialogoController.cs
JugadorController.cs	143	Modelos/Jugador/JugadorController.cs
TriggerCuerpoUI.cs	57	Modelos/Jugador/TriggerCuerpoUI.cs
MinijuegoCerebroController.cs	223	Modelos/Minijuego/Cabeza/MinijuegoCerebroController.cs
MinijuegoCantoController.cs	310	Modelos/Minijuego/Cantar/MinijuegoCantoController.cs
MinijuegoSaludoController.cs	167	Modelos/Minijuego/Mano/MinijuegoSaludoController.cs

MinijuegoSenalControl ler.cs	246	Modelos/Minijuego/Mano/MinijuegoSenalController.cs
PanelManoController. cs	92	Modelos/Minijuego/Mano/PanelManoController.cs
MinijuegoPechoContro ller.cs	172	Modelos/Minijuego/Pecho/MinijuegoPechoController.cs
MinijuegoPiernaContr oller.cs	362	Modelos/Minijuego/Pierna/MinijuegoPiernaController.cs
MinijuegoResponderC ontroller.cs	198	Modelos/Minijuego/boca/MinijuegoResponderController.cs
PanelBocaController.c s	48	Modelos/Minijuego/boca/PanelBocaController.cs
TransicionZonaControl ler.cs	157	Modelos/transicion/TransicionZonaController.cs

CamaraController.cs

20 líneas

Scripts/Modelos/Camara/CamaraController.cs

Descripcion	Controlador de seguimiento suave de camara. En LateUpdate() interpola la posición de la camara hacia el jugador usando Lerp con velocidadSmooth. Solo opera cuando CamaraData.seguirJugador es true, lo que permite inmovilizarla durante diálogos. Dependencias: CamaraData.
--------------------	---

```
using UnityEngine;
[RequireComponent(typeof(CamaraData))]
public class CamaraController : MonoBehaviour
{
    private CamaraData _data;
    private void Awake()
    {
        _data = GetComponent<CamaraData>();
    }
    private void LateUpdate()
    {
        if (!_data.seguirJugador || _data.jugador == null) return;
        float nuevoX = Mathf.Lerp(transform.position.x, _data.jugador.position.x, _data.velocidadSmooth * Time.deltaTime);
        transform.position = new Vector3(nuevoX, transform.position.y, transform.position.z);
    }
}
```

CuerpoUIController.cs

151 líneas

Scripts/Modelos/Cuerpo/CuerpoUIController.cs

Descripcion	Controlador del panel de selección de partes del cuerpo. Itera ParteCuerpoData[] para registrar listeners en cada botón. Al pulsar, verifica JugadorHabilidadesData y dispara el evento estático OnParteSeleccionada(ParteCuerpo) que MinijuegoManager escucha. Aplica colores de estado bloqueado/desbloqueado. Dependencias: CuerpoUIData, JugadorHabilidadesData, MinijuegoManager.
--------------------	--

```
using UnityEngine;
/// <summary>
/// Controlador de la UI del cuerpo.
/// Abre el panel al llamarse desde el trigger del jugador,
/// activa o desactiva botones según las habilidades desbloqueadas,
/// y notifica que parte fue seleccionada.
/// </summary>
[RequireComponent(typeof(CuerpoUIData))]
public class CuerpoUIController : MonoBehaviour
```



```
//
#region Botones
/// <summary>
/// Recorre el array de partes, activa o desactiva cada boton
/// segun el estado de habilidades del jugador, y asigna listeners.
/// </summary>
private void ConfigurarBotones()
{
    if (_data.partes == null)
        return;
    foreach (ParteCuerpoData entrada in _data.partes)
    {
        if (entrada == null || entrada.boton == null)
        {
            Debug.LogWarning("[CuerpoUIController] Entrada de parte o boton nulo. Revisar Inspector.");
            continue;
        }
        bool desbloqueada = _habilidades.EstaDesbloqueada(entrada.parte);
        entrada.boton.interactable = desbloqueada;
        if (entrada.imagenBoton != null)
            entrada.imagenBoton.color = desbloqueada ? _data.colorDesbloqueado : _data.colorBloqueado;
        entrada.boton.onClick.RemoveAllListeners();
        if (desbloqueada)
        {
            ParteCuerpo parteCapturada = entrada.parte;
            entrada.boton.onClick.AddListener(() => AlSeleccionarParte(parteCapturada));
        }
    }
}

private void LimpiarBotones()
{
    if (_data.partes == null)
        return;
    foreach (ParteCuerpoData entrada in _data.partes)
    {
        if (entrada != null && entrada.boton != null)
            entrada.boton.onClick.RemoveAllListeners();
    }
}

private void AlSeleccionarParte(ParteCuerpo parte)
{
    CerrarPanel();
    OnParteSeleccionada?.Invoke(parte);
}
#endregion
}
```

DialogController.cs

160 lines

Scripts/Modelos/Dialogo/DialogoController.cs

Descripción	Controlador del sistema de diálogos. Iniciar(DialogoData) abre el panel y muestra la primera línea. Cada vez que el jugador presiona la tecla de avance (definida en DialogUIData) pasa a la siguiente DialogoLinea. Al terminar cierra el panel y dispara el evento estatico OnDialogoTerminado. Dependencias: DialogUIData, DialogoData, DialogoLinea.
--------------------	--

[illegible]

```
// Estado
private DialogoData _dialogoActual;
private int _indiceActual;
private bool _dialogoActivo;

// Evento
/// <summary>
/// Se dispara cuando el dialogo actual termina completamente.
/// Cualquier sistema que inicie un dialogo puede suscribirse aqui.
/// </summary>
public static event System.Action OnDialogoTerminado;

//
#region Unity Callbacks
private void Awake()
{
    _data = GetComponent<DialogoUIData>();
}

private void Update()
{
    if (!_dialogoActivo) return;
    if (Input.GetKeyDown(_data.teclaAvanzar))
        Avanzar();
}

#endregion

//
#region API Publica
/// <summary>
/// Inicia un dialogo dado un DialogoData.
/// Si hay un dialogo activo lo interrumpe y comienza el nuevo.
/// </summary>
public void Iniciar(DialogoData dialogo)
{
    if (dialogo == null)
    {
        Debug.LogWarning("[DialogoController] Se intento iniciar un dialogo nulo.");
        return;
    }

    if (dialogo.lineas == null || dialogo.lineas.Length == 0)
    {
        Debug.LogWarning("[DialogoController] El dialogo no tiene lineas.");
        return;
    }

    _dialogoActual = dialogo;
    _indiceActual = 0;
    _dialogoActivo = true;
    _data.panelDialogo.SetActive(true);
    MostrarLineaActual();
}

#endregion

//
#region Logica
/// <summary>
/// Avanza a la siguiente linea o termina el dialogo.
/// </summary>
private void Avanzar()
{
    _indiceActual++;
    if (_indiceActual >= _dialogoActual.lineas.Length)
    {
        Terminar();
        return;
    }

    MostrarLineaActual();
}

/// <summary>
/// Muestra la linea actual en la UI.
/// </summary>
```

```

private void MostrarLineaActual()
{
    if (!ValidarReferenciasUI()) return;
    DialogoLinea linea = _dialogoActual.lineas[_indiceActual];
    // Texto
    _data.textoDialogo.text = linea.texto;
    // Imagenes – se muestra solo la del lado activo
    if (linea.esIzquierda)
    {
        MostrarImagen(_data.imagenIzquierda, linea.imagen);
        OcultarImagen(_data.imagenDerecha);
    }
    else
    {
        MostrarImagen(_data.imagenDerecha, linea.imagen);
        OcultarImagen(_data.imagenIzquierda);
    }
}

private void MostrarImagen(UnityEngine.UI.Image imagen, Sprite sprite)
{
    if (imagen == null) return;
    imagen.sprite = sprite;
    imagen.gameObject.SetActive(sprite != null);
}

private void OcultarImagen(UnityEngine.UI.Image imagen)
{
    if (imagen == null) return;
    imagen.gameObject.SetActive(false);
}

/// <summary>
/// Cierra el panel y dispara el evento de fin.
/// </summary>
private void Terminar()
{
    _dialogoActivo = false;
    _dialogoActual = null;
    CerrarPanel();
    OnDialogoTerminado?.Invoke();
}

private void CerrarPanel()
{
    if (_data != null && _data.panelDialogo != null)
        _data.panelDialogo.SetActive(false);
}

private bool ValidarReferenciasUI()
{
    if (_data.textoDialogo == null)
    {
        Debug.LogError("[DialogoController] textoDialogo no asignado en DialogoUIData.");
        return false;
    }
    return true;
}

#endregion
}

```

JugadorController.cs

143 líneas

Scripts/Modelos/Jugador/JugadorController.cs

Descripcion

Controlador principal del jugador. En Update() lee el input WASD/flechas, construye direccionInput y lo aplica a la velocidad del Rigidbody usando aceleracion/desaceleracion de JugadorData. Actualiza animaciones del Animator y el SpriteRenderer (flip). Respeta puedeMoveirse. Gestiona audio de caminata. Dependencias: JugadorData.

```

using UnityEngine;
/// <summary>

```

[illegible]

```

///
/// Por que es frame-rate independent:
/// Todo el calculo vive en FixedUpdate, que Unity llama a intervalos
/// fijos (Time.fixedDeltaTime). MoveTowards avanza la misma cantidad
/// de unidades por segundo sin importar cuantos frames renders haya.
/// </summary>
private void AplicarMovimiento()
{
    // Velocidad actual solo en el plano XZ (ignoramos Y para no
    // interferir con la gravedad ni con saltos futuros)
    Vector3 velActualXZ = new Vector3(_rb.linearVelocity.x, 0f, _rb.linearVelocity.z);
    Vector3 velObjetivo;
    float tasa;
    if (_data.direccionInput != Vector3.zero)
    {
        // Hay input → acelerar hacia la direccion deseada
        velObjetivo = _data.direccionInput * _data.velocidadMaxima;
        tasa = _data.aceleracion * Time.fixedDeltaTime;
    }
    else
    {
        // Sin input → frenar hasta cero
        velObjetivo = Vector3.zero;
        tasa = _data.desaceleracion * Time.fixedDeltaTime;
    }

    // MoveTowards nunca supera el target ni la tasa: sin overshooting
    Vector3 nuevaVelXZ = Vector3.MoveTowards(velActualXZ, velObjetivo, tasa);
    // Reacomponemos con la Y original para respetar gravedad / saltos
    _rb.linearVelocity = new Vector3(nuevaVelXZ.x, _rb.linearVelocity.y, nuevaVelXZ.z);
    bool moviendose = _data.direccionInput != Vector3.zero;
    // Animacion
    _data.animador.Play(moviendose ? _data.animCaminando : _data.animQuieto);
    // Audio
    if (moviendose)
    {
        if (!_data.audioCaminata.isPlaying)
        _data.audioCaminata.Play();
    }
    else
    {
        _data.audioCaminata.Stop();
    }

    // Flip X
    if (_data.direccionInput.x != 0)
    _data.spriteRenderer.flipX = _data.direccionInput.x < 0;
    }
}
#endregion
}

```

TriggerCuerpoUI.cs

57 lineas

Scripts/Modelos/Jugador/TriggerCuerpoUI.cs

[illegible]

MinijuegoCerebroController.cs

223 lines

Scripts/Modelos/Minijuego/Cabeza/MinijuegoCerebroController.cs

Descripción	Controlador del minijuego de cabeza (cerebro). Iniciar() activa el panel y asigna listeners de click a los distractores. En Update() mueve los distractores hacia el cerebro, detecta colisiones por distancia y reduce el alpha del cerebro. El jugador presiona espacio para recuperar alpha. Al ganar/perder reporta el resultado via MisionesGlobal. Dependencias: MinijuegoCerebroData, DistractorCerebro.
--------------------	---


```

    if (_data.panelCerebro == null)
    {
        Debug.LogError("[MinijuegoCerebroController] panelCerebro no asignado.");
        return false;
    }
    if (_data.imagenCerebro == null)
    {
        Debug.LogError("[MinijuegoCerebroController] imagenCerebro no asignada.");
        return false;
    }
    if (_data.rectCerebro == null)
    {
        Debug.LogError("[MinijuegoCerebroController] rectCerebro no asignado.");
        return false;
    }
    if (_data.distractores == null || _data.distractores.Length != 4)
    {
        Debug.LogError("[MinijuegoCerebroController] Se necesitan exactamente 4 distractores.");
        return false;
    }
    return true;
}

#endregion
}

```

MinijuegoCantoController.cs

310 lines

Scripts/Modelos/Minijuego/Cantar/MinijuegoCantoController.cs

Descripcion	Controlador del minijuego de canto. Gestiona el pool de notas por carril: en cada intervalo activa una nota aleatoria y la mueve hacia abajo. Al pulsar un boton de carril evalua si alguna nota activa esta dentro del margen de acierto. Cuenta aciertos y fallos; al alcanzar los limites reporta victoria o derrota. Dependencias: MinijuegoCantoData, NotaCanto.
--------------------	---


```
{
    if (_data.panelCanto == null)
    {
        Debug.LogError("[MinijuegoCantoController] panelCanto no asignado.");
        return false;
    }
    if (_data.botones == null || _data.botones.Length != 4)
    {
        Debug.LogError("[MinijuegoCantoController] Se necesitan exactamente 4 botones.");
        return false;
    }
    return true;
}

#endregion
}
```

MinijuegoSaludoController.cs

167 lines

Scripts/Modelos/Minijuego/Mano/MinijuegoSaludoController.cs

Descripcion	Controlador del minijuego de saludo de mano. Al iniciar elige aleatoriamente si la mano del otro personaje aparece arriba o abajo, coloca las imagenes en los botones correspondientes y empieza la cuenta regresiva de la barra. Si el jugador presiona el boton correcto dentro del tiempo muestra el exito. Dependencias: MinijuegoSaludoData.
--------------------	---


```
}  
#endregion  
// ████████████████████████████████████████████████████████████████████████████  
#region Helpers  
private void LimpiarEstado()  
{  
    _data.indiceActual = 0;  
    _esperandoRespuesta = false;  
}  
  
private void LimpiarBotones()  
{  
    if (_botonesActivos == null) return;  
    foreach (Button b in _botonesActivos)  
        if (b != null) b.onClick.RemoveAllListeners();  
}  
  
private bool ValidarReferencias()  
{  
    if (_data.panelSenal == null)  
    {  
        Debug.LogError("[MinijuegoSenalController] panelSenal no asignado.");  
        return false;  
    }  
    if (!_data.esCompleja && (_data.botones == null || _data.botones.Length == 0))  
    {  
        Debug.LogError("[MinijuegoSenalController] botones simples no asignados.");  
        return false;  
    }  
    if (_data.esCompleja && (_data.botonesComplejos == null || _data.botonesComplejos.Length == 0))  
    {  
        Debug.LogError("[MinijuegoSenalController] botonesComplejos no asignados.");  
        return false;  
    }  
    return true;  
}  
#endregion  
}
```

PanelManoController.cs

92 lineas

Scripts/Modelos/Minijuego/Mano/PanelManoController.cs

Descripcion	Controlador del panel de seleccion de la mano. Al abrir el panel registra listeners en botonSaludo y botonSenal. Al pulsar dispara el evento estatico correspondiente (OnSaludoSeleccionado / OnSenalSeleccionada) que MinijuegoManager escucha. Dependencias: PanelManoData, MinijuegoManager.
--------------------	---

```
using UnityEngine;

/// <summary>
/// Controlador del panel intermedio de la mano.
/// Muestra dos opciones al jugador: Saludo o Senal.
/// </summary>
[RequireComponent(typeof(PanelManoData))]
public class PanelManoController : MonoBehaviour
{
    // ■■ Referencias ████████████████████████████████████████████████████
    private PanelManoData _data;

    // ■■ Eventos ████████████████████████████████████████████████████████

    public static event System.Action OnSaludoSeleccionado;
    public static event System.Action OnSenalSeleccionada;

    // ████████████████████████████████████████████████████████████████████

    #region Unity Callbacks
    private void Awake()
    {
        _data = GetComponent<PanelManoData>();
    }

    private void Start()
    {
```

[illegible]

MinijuegoPechoController.cs

172 lines

Scripts/Modelos/Minijuego/Pecho/MinijuegoPechoController.cs

Descripcion	Controlador del minijuego de ritmo del pecho. Cada ronda inicia el circulo externo en tamanoInicial y lo reduce cada frame a velocidadActual. Al pulsar el boton del corazon compara tamanoActual con tamanoCorazon $\pm$ margenAcuerdo. Acuerdo: siguiente ronda. Fallo: penalizacion. Al terminar todas las rondas reporta el resultado. Dependencias: MinijuegoPechoData.
--------------------	--

```
using UnityEngine;
using System.Collections;

/// <summary>
/// Controlador del minijuego de ritmo del pecho.
/// Un circulo externo se achica hacia el corazon.
```

```
// El jugador debe hacer click cuando coinciden.  
// Exito N veces → valor 1. Fallo → valor 0.  
// </summary>  
[RequireComponent(typeof(MinijuegoPechoData))]  
public class MinijuegoPechoController : MonoBehaviour  
{  
    private MinijuegoPechoData _data;  
    // ████████████████████████████████████████████████████████████████████████████  
#region Unity Callbacks  
    private void Awake()  
    {  
        _data = GetComponent<MinijuegoPechoData>();  
    }  
    private void Update()  
    {  
        if (!_data.esperandoClick) return;  
        // Achicar el circulo  
        _data.tamanoActual -= _data.velocidadActual * Time.deltaTime;  
        // Aplicar tamano al circulo externo  
        _data.circuloExterno.sizeDelta = new Vector2(_data.tamanoActual, _data.tamanoActual);  
        // Si el circulo paso el tamano minimo sin click → fallo  
        if (_data.tamanoActual <= _data.tamanoCorazon - _data.margenAcierto)  
        {  
            _data.esperandoClick = false;  
            Debug.Log("[MinijuegoPecho] Fallo - click tardio o no presionado");  
            StartCoroutine(Terminar(false));  
        }  
    }  
#endregion  
    // ████████████████████████████████████████████████████████████████████████████  
#region API Publica  
    public void Iniciar()  
    {  
        if (!ValidarReferencias()) return;  
        LimpiarEstado();  
        _data.panelPecho.SetActive(true);  
        _data.botonCorazon.onClick.RemoveAllListeners();  
        _data.botonCorazon.onClick.AddListener(AlClickearCorazon);  
        IniciarRonda();  
    }  
#endregion  
    // ████████████████████████████████████████████████████████████████████████████  
#region Logica de Ronda  
    private void IniciarRonda()  
    {  
        _data.tamanoActual = _data.tamanoInicial;  
        _data.velocidadActual = Random.Range(_data.velocidadMin, _data.velocidadMax);  
        _data.esperandoClick = true;  
        _data.circuloExterno.sizeDelta = new Vector2(_data.tamanoActual, _data.tamanoActual);  
        Debug.Log($"[MinijuegoPecho] Ronda {_data.rondaActual + 1}/{_data rondasTotales} " +  
            $"- velocidad={_data.velocidadActual:F1}");  
    }  
    private void AlClickearCorazon()  
    {  
        if (!_data.esperandoClick) return;  
        float diferencia = Mathf.Abs(_data.tamanoActual - _data.tamanoCorazon);  
        if (diferencia <= _data.margenAcierto)  
        {  
            // Acierto  
            _data.esperandoClick = false;  
            _data.rondaActual++;  
            Debug.Log($"[MinijuegoPecho] Acierto ronda {_data.rondaActual}/{_data rondasTotales} " +  
                $"- diferencia={diferencia:F1}");  
            if (_data.rondaActual >= _data.rondasTotales)  
            {  
                StartCoroutine(Terminar(true));  
            }  
        }  
    }  
}
```


MinijuegoPiernaController.cs

362 lineas

Scripts/Modelos/Minijuego/Pierna/MinijuegoPiernaController.cs

Descripcion	Controlador del minijuego de ensamblaje de pierna. Genera un orden aleatorio de Cadera→Rodilla→Pie al iniciar y muestra el objetivo en el panel lateral. El jugador hace click en celdas para colocar las partes en el orden correcto. Al colocar la parte equivocada limpia la celda. Al completar el orden correcto reporta victoria. Dependencias: MinijuegoPiernaData, CeldaCuadriculaData.
--------------------	---

[illegible]

```

    /// <summary>
    /// Limpia el estado de celdas y variables del turno anterior.
    /// </summary>
    private void LimpiarEstado()
    {
        _data.turnoActual = 0;
        _posicionCadera = null;
        _posicionRodilla = null;
        _posicionPie = null;
        foreach (CeldaCuadrriculaData celda in _data.celdas)
        {
            if (celda != null)
            {
                celda.Limpiar();
                // Agregar estas dos lineas:
                if (celda.boton != null)
                {
                    celda.boton.gameObject.SetActive(true);
                    celda.boton.interactable = true;
                }
            }
        }
    }

    /// <summary>
    /// Mezcla las 3 partes en orden aleatorio usando Fisher-Yates.
    /// </summary>
    private void GenerarOrdenAleatorio()
    {
        PartePierna[] partes = { PartePierna.Cadera, PartePierna.Rodilla, PartePierna.Pie };
        for (int i = partes.Length - 1; i > 0; i--)
        {
            int j = Random.Range(0, i + 1);
            PartePierna temp = partes[i];
            partes[i] = partes[j];
            partes[j] = temp;
        }
        _data.ordenAleatorio = partes;
    }

    /// <summary>
    /// Asigna los sprites al panel de orden y los hace visibles.
    /// </summary>
    private void ConfigurarImagenesOrden()
    {
        for (int i = 0; i < 3; i++)
        {
            UnityEngine.UI.Image img = _data.ObtenerImagenOrden(i);
            if (img == null) continue;
            Sprite sprite = _data.ObtenerSprite(_data.ordenAleatorio[i]);
            img.sprite = sprite;
            img.gameObject.SetActive(true);
            // Transparencia completa = visible
            Color c = img.color;
            c.a = 1f;
            img.color = c;
        }
    }

    /// <summary>
    /// Asigna listeners a los botones de cada celda.
    /// </summary>
    private void ConfigurarBotonesCeldas()
    {
        foreach (CeldaCuadrriculaData celda in _data.celdas)
        {
            if (celda == null || celda.boton == null) continue;
            celda.boton.onClick.RemoveAllListeners();
            // Captura local para el lambda
            CeldaCuadrriculaData celdaCapturada = celda;

```

```
celda.boton.onClick.AddListener(() => AlClickearCelda(celdaCapturada));  
}  
  
private void OcultarConexiones()  
{  
if (_data.lineaCaderaRodilla != null)  
_data.lineaCaderaRodilla.gameObject.SetActive(false);  
if (_data.lineaRodillaPie != null)  
_data.lineaRodillaPie.gameObject.SetActive(false);  
}  
  
#endregion  
  
// ████████████████████████████████████████████████████████████████████████  
  
#region Logica de Clicks  
private void AlClickearCelda(CeldaCuadrículaData celda)  
{  
if (celda == null) return;  
if (celda.EstaOcupada) return;  
if (_data.turnoActual >= 3) return;  
PartePierna parteActual = _data.ordenAleatorio[_data.turnoActual];  
    // Colocar parte en la celda  
ColocarParteEnCelda(celda, parteActual);  
    // Desvanecer imagen del panel de orden  
DesvanecerImagenOrden(_data.turnoActual);  
    // Guardar posicion para conexiones  
GuardarPosicion(parteActual, celda.GetComponent<RectTransform>());  
    // Actualizar conexiones visuales  
ActualizarConexiones();  
_data.turnoActual++;  
    // Si ya se colocaron las 3 partes, esperar y reportar  
if (_data.turnoActual >= 3)  
StartCoroutine(EsperarYReportar());  
}  
  
private void ColocarParteEnCelda(CeldaCuadrículaData celda, PartePierna parte)  
{  
celda.parteOcupante = parte;  
if (celda.imagenParte != null)  
{  
celda.imagenParte.sprite = _data.ObtenerSprite(parte);  
celda.imagenParte.gameObject.SetActive(true);  
}  
}  
  
private void DesvanecerImagenOrden(int indice)  
{  
UnityEngine.UI.Image img = _data.ObtenerImagenOrden(indice);  
if (img == null) return;  
Color c = img.color;  
c.a = 0.25f;  
img.color = c;  
}  
  
private void GuardarPosicion(PartePierna parte, RectTransform rect)  
{  
switch (parte)  
{  
case PartePierna.Cadera: _posicionCadera = rect; break;  
case PartePierna.Rodilla: _posicionRodilla = rect; break;  
case PartePierna.Pie: _posicionPie = rect; break;  
}  
}  
  
#endregion  
  
// ████████████████████████████████████████████████████████████████████████  
  
#region Conexiones Visuales  
    /// <summary>  
    /// Actualiza posicion y visibilidad de las lineas de conexion  
    /// segun que partes ya han sido colocadas.  
    /// Usa el punto medio entre dos RectTransform como posicion de la linea  
    /// y rota la imagen para apuntar de uno al otro.
```



```
ActualizarBarra();  
DetectarAyuda();  
}  
  
#endregion  
  
// ████████████████████████████████████████████████████████████████████████████  
  
#region API Publica  
  
public void Iniciar()  
{  
if (!ValidarReferencias()) return;  
LimpiarEstado();  
ConfigurarContenido();  
ConfigurarBotones();  
ResetearAyuda();  
_data.panelResponder.SetActive(true);  
_data.esperandoRespuesta = true;  
}  
  
#endregion  
  
// ████████████████████████████████████████████████████████████████████████████  
  
#region Update  
  
private void ActualizarBarra()  
{  
    _data.tiempoRestante -= Time.deltaTime * _data.multiplicadorVelocidad;  
    float t = Mathf.Clamp01(_data.tiempoRestante / _data.tiempoLmite);  
    float nuevoAncho = _data.anchoBarraInicial * t;  
    _data.barraTiempo.sizeDelta = new Vector2(nuevoAncho, _data.barraTiempo.sizeDelta.y);  
    if (_data.tiempoRestante <= 0f)  
    {  
        _data.esperandoRespuesta = false;  
        Debug.Log("[MinijuegoResponder] Fallo - tiempo agotado");  
        StartCoroutine(Terminar(false));  
    }  
}  
  
private void DetectarAyuda()  
{  
    if (Input.GetKeyDown(KeyCode.Space))  
    {  
        // Transparentar imagen de ayuda  
        if (_data.imagenAyuda != null)  
        {  
            Color c = _data.imagenAyuda.color;  
            c.a = Mathf.Max(0f, c.a - _data.alphaAyudaPorEspacio);  
            _data.imagenAyuda.color = c;  
        }  
        // Acelerar barra  
        _data.multiplicadorVelocidad *= _data.aceleracionBarraPorEspacio;  
        Debug.Log($"[MinijuegoResponder] Ayuda usada - " +  
            $"alphaAyuda={_data.imagenAyuda?.color.a:F2} " +  
            $"multiplicador={_data.multiplicadorVelocidad:F2}");  
    }  
}  
  
#endregion  
  
// ████████████████████████████████████████████████████████████████████████████  
  
#region Configuración  
  
private void ConfigurarContenido()  
{  
    if (_data.textoBoton0 != null) _data.textoBoton0.text = _data.textoBotones[0];  
    if (_data.textoBoton1 != null) _data.textoBoton1.text = _data.textoBotones[1];  
    if (_data.textoBoton2 != null) _data.textoBoton2.text = _data.textoBotones[2];  
    if (_data.textoAyuda != null)  
        _data.textoAyuda.text = _data.textoAyudaContenido;  
}  
  
private void ConfigurarBotones()  
{  
    LimpiarBotones();  
    if (_data.boton0 != null) _data.boton0.onClick.AddListener(() => AlResponder(0));  
    if (_data.boton1 != null) _data.boton1.onClick.AddListener(() => AlResponder(1));  
}
```

```
if (_data.boton2 != null) _data.boton2.onClick.AddListener(() => AlResponder(2));  
}  
  
private void ResetearAyuda()  
{  
    if (_data.imagenAyuda != null)  
    {  
        Color c = _data.imagenAyuda.color;  
        c.a = 1f;  
        _data.imagenAyuda.color = c;  
    }  
}  
  
#endregion  
  
// ████████████████████████████████████████████████████████████████████████████████  
  
#region Logica  
private void AlResponder(int indice)  
{  
    if (!_data.esperandoRespuesta) return;  
    _data.esperandoRespuesta = false;  
    bool correcto = indice == _data.indiceRespuestaCorrecta  
    && _data.indiceRespuestaCorrecta >= 0  
    && _data.indiceRespuestaCorrecta <= 2;  
    Debug.Log($"[MinijuegoResponder] Boton {indice} presionado - " +  
    $"correcto={correcto}");  
    StartCoroutine(Terminar(correcto));  
}  
  
private IEnumerator Terminar(bool acerto)  
{  
    LimpiarBotones();  
    yield return new WaitForSeconds(0.4f);  
    _data.panelResponder.SetActive(false);  
    int valor = acerto ? 1 : 0;  
    Debug.Log($"[MinijuegoResponder] Resultado → TipoAccion.Boca valor={valor}");  
    AccionResultado resultado = new AccionResultado(TipoAccion.Boca, valor);  
    if (MisionesGlobal.Instancia != null)  
        MisionesGlobal.Instancia.ReportarResultado(resultado);  
    else  
        Debug.LogWarning("[MinijuegoResponderController] MisionesGlobal no encontrado.");  
}  
  
#endregion  
  
// ████████████████████████████████████████████████████████████████████████████████  
  
#region Helpers  
private void LimpiarEstado()  
{  
    _data.tiempoRestante = _data.tiempoLimite;  
    _data.multiplicadorVelocidad = 1f;  
    _data.esperandoRespuesta = false;  
    if (_data.barraTiempo != null)  
        _data.barraTiempo.sizeDelta = new Vector2(_data.anchoBarraInicial, _data.barraTiempo.sizeDelta.y);  
}  
  
private void LimpiarBotones()  
{  
    if (_data.boton0 != null) _data.boton0.onClick.RemoveAllListeners();  
    if (_data.boton1 != null) _data.boton1.onClick.RemoveAllListeners();  
    if (_data.boton2 != null) _data.boton2.onClick.RemoveAllListeners();  
}  
  
private bool ValidarReferencias()  
{  
    if (_data.panelResponder == null)  
    {  
        Debug.LogError("[MinijuegoResponderController] panelResponder no asignado.");  
        return false;  
    }  
    if (_data.barraTiempo == null)  
    {  
        Debug.LogError("[MinijuegoResponderController] barraTiempo no asignada.");  
        return false;  
    }  
}
```

```

return true;
}
#endregion
}

```

PanelBocaController.cs

48 líneas

Scripts/Modelos/Minijuego/boca/PanelBocaController.cs

Descripción	Controlador del panel de selección de la boca. Muestra dos opciones al jugador: Responder o Cantar. Al pulsar, dispara el evento estático correspondiente (OnResponderSeleccionado / OnCantarSeleccionado) que MinijuegoManager escucha. Dependencias: PanelBocaData, MinijuegoManager.
--------------------	---

```

using UnityEngine;
/// <summary>
/// Controlador del panel intermedio de la boca.
/// </summary>
[RequireComponent(typeof(PanelBocaData))]
public class PanelBocaController : MonoBehaviour
{
    private PanelBocaData _data;
    public static event System.Action OnResponderSeleccionado;
    public static event System.Action OnCantarSeleccionado;
    private void Awake()
    {
        _data = GetComponent<PanelBocaData>();
    }
    private void Start()
    {
        CerrarPanel();
    }
    public void AbrirPanel()
    {
        if (_data.panelBoca == null) { Debug.LogError("[PanelBocaController] panelBoca no asignado."); return; }
        LimpiarBotones();
        if (_data.botonResponder != null) _data.botonResponder.onClick.AddListener(AlSeleccionarResponder);
        if (_data.botonCantar != null) _data.botonCantar.onClick.AddListener(AlSeleccionarCantar);
        _data.panelBoca.SetActive(true);
    }
    public void CerrarPanel()
    {
        if (_data.panelBoca == null) return;
        _data.panelBoca.SetActive(false);
        LimpiarBotones();
    }
    private void LimpiarBotones()
    {
        if (_data.botonResponder != null) _data.botonResponder.onClick.RemoveAllListeners();
        if (_data.botonCantar != null) _data.botonCantar.onClick.RemoveAllListeners();
    }
    private void AlSeleccionarResponder() { CerrarPanel(); OnResponderSeleccionado?.Invoke(); }
    private void AlSeleccionarCantar() { CerrarPanel(); OnCantarSeleccionado?.Invoke(); }
}

```

TransicionZonaController.cs

157 líneas

Scripts/Modelos/transicion/TransicionZonaController.cs

Descripción	Controlador de la transición entre zonas de escena mediante un cuadro negro. Al activarse baja el cuadro negro (fade out), desactiva la zona de salida, activa la zona de entrada, teletransporta al jugador y a la cámara a los puntos destino, y luego sube el cuadro (fade in). Opcionalmente cambia el audio de caminata y reactiva el follow de cámara. Dependencias: TransicionZonaData, CamaraData.
--------------------	--

```

using UnityEngine;
using System.Collections;
/// <summary>
/// Trigger de transición entre zonas.

```



```

}
#endregion
// ████████████████████████████████████████████████████████████████████████████
#region Transicion
private IEnumerator EjecutarTransicion(Transform jugador)
{
    _enTransicion = true;
    // 1 - Bloquear movimiento
    _jugadorData.puedeMoverse = false;
    if (_data.audioTransicion != null)
        _data.audioTransicion.Play();
    // 2 - Bajar cuadro negro hasta Y = 0
    yield return StartCoroutine(MoverCuadro(0f));
    // 3 - Teletransportar jugador y camara
    jugador.position = _data.puntoJugador.position;
    _data.camara.position = _data.puntoCamara.position;
    // 4 - Intercambiar zonas
    if (_data.zonaEntrada != null)
        _data.zonaEntrada.SetActive(true);
    if (_data.zonaSalida != null)
        _data.zonaSalida.SetActive(false);
    if (_data.nuevaCaminata != null && _jugadorData.audioCaminata != null)
        _jugadorData.audioCaminata.clip = _data.nuevaCaminata;
    CamaraData camaraData = _data.camara.GetComponent<CamaraData>();
    if (camaraData != null)
        camaraData.seguirJugador = _data.SeguirJugadorSiguiente;
    // 5 - Subir cuadro negro hasta Y = 2
    yield return StartCoroutine(MoverCuadro(2f));
    // 6 - Reactivar movimiento
    _jugadorData.puedeMoverse = true;
    _enTransicion = false;
}

/// <summary>
/// Mueve el cuadro negro suavemente hasta el Y destino.
/// </summary>
private IEnumerator MoverCuadro(float yDestino)
{
    Vector3 posInicial = _data.cuadroNegro.localPosition;
    Vector3 posDestino = new Vector3(posInicial.x, yDestino, posInicial.z);
    float distancia = Mathf.Abs(posInicial.y - yDestino);
    if (distancia < 0.001f) yield break;
    float tiempoTotal = distancia / _data.velocidadTransicion;
    float tiempoTranscurrido = 0f;
    while (tiempoTranscurrido < tiempoTotal)
    {
        tiempoTranscurrido += Time.deltaTime;
        float t = Mathf.Clamp01(tiempoTranscurrido / tiempoTotal);
        _data.cuadroNegro.localPosition = Vector3.Lerp(posInicial, posDestino, t);
        yield return null;
    }
    _data.cuadroNegro.localPosition = posDestino;
}
#endregion
}

```